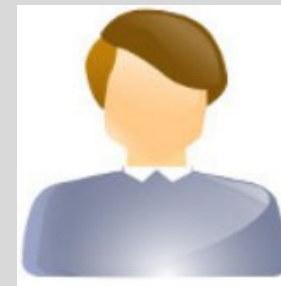
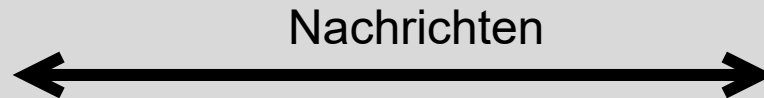




Informatik 2: Kryptographie 2/2



Alice



Bob

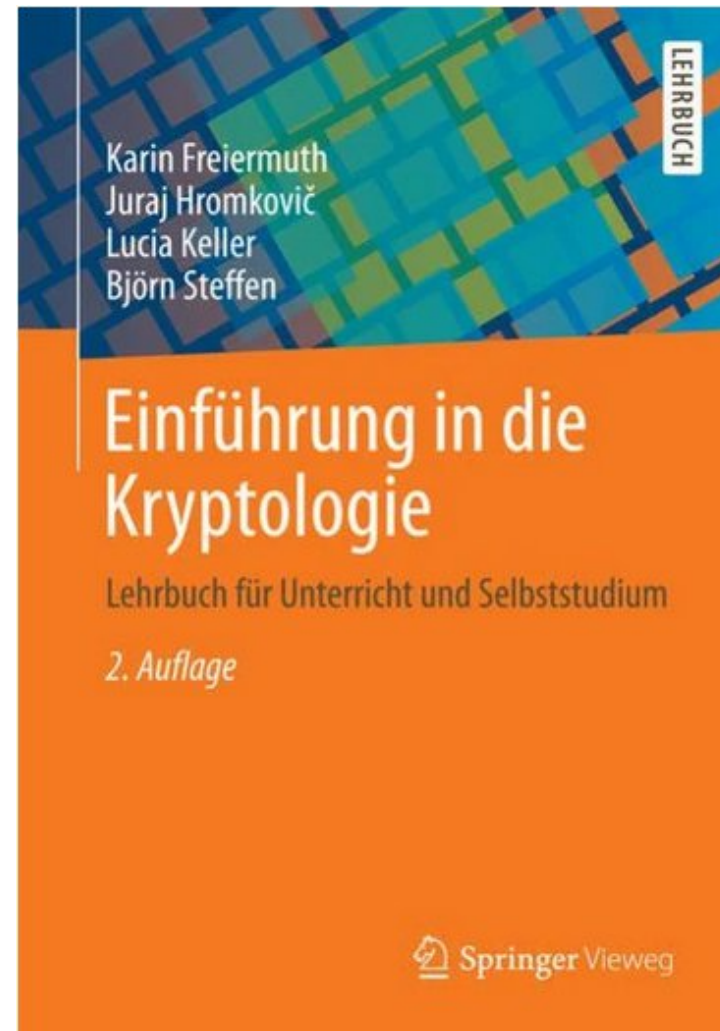
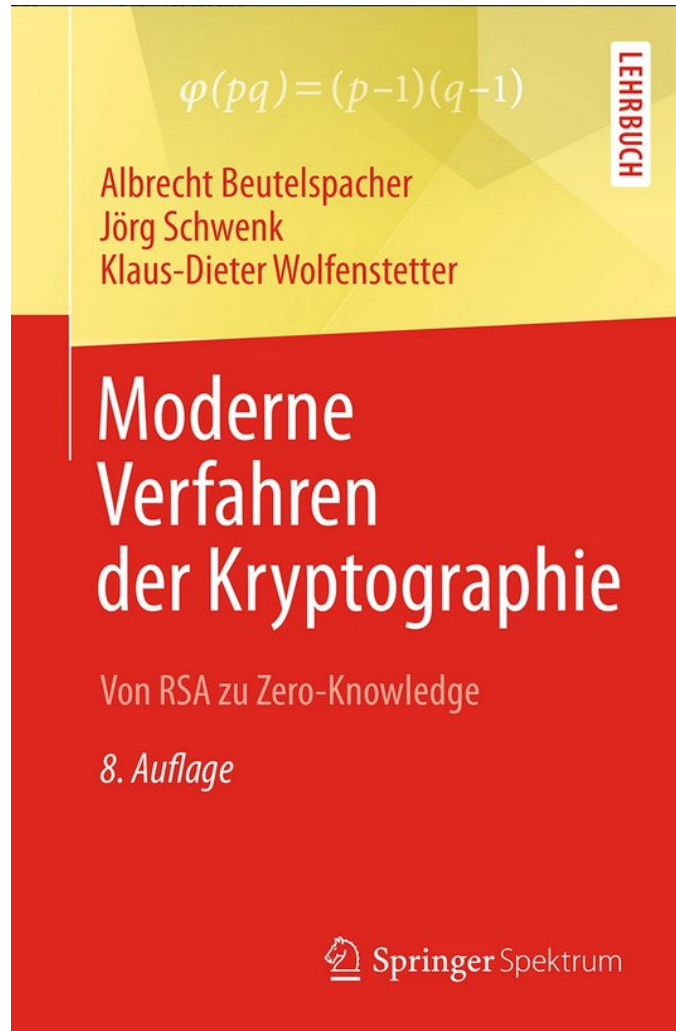


Bisher

- Alice & Bob
- Transpositionsverfahren
(SKYTALE, Fleißnersche Schablone)
- Steganographie
(RICHELIEU)
- Substitutionsverfahren
 - Monoalphabetische
(CAESAR, POLYBIOS, FREIMAURER)
 - Polyalphabetische
(VIGENÈRE, ENIGMA, One-time-pad)
- Mathematische Grundlagen
der modernen Kryptographie
 - Modulares Rechnen

Diese Woche

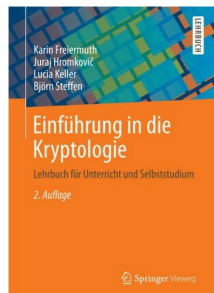
- Moderne Verfahren der Kryptographie
 - Modulares Rechnen
 - Symmetrische Verfahren: DES & AES
 - Schlüsseltausch: Diffie-Hellman Protokoll
 - Asymmetrische Verfahren: RSA
- Bitcoins



Modulares Rechnen ist die Basis der modernen Kryptologie



Modulares Rechnen mit der algebraische Struktur $(\mathbb{Z}_a, \oplus_a, \odot_a)$ und $\mathbb{Z}_a = \{0, 1, \dots, a - 1\}$



Modulares Rechnen: Übersicht



Modulare Addition $\oplus$:

Es gilt folgende Regel:

Inverse Element in $\oplus_a$:

$$x \oplus_a y = (x + y) \bmod a$$

$$x \oplus_a y = ((x \bmod a) + (y \bmod a)) \bmod a$$

$$(-x) \bmod a = (a - x) \bmod a$$

Modulare Subtraktion $\ominus$:

Es gelten folgende Regeln:

$$x \ominus_a y = (x - y) \bmod a$$

$$x \ominus_a y = ((x \bmod a) - (y \bmod a)) \bmod a$$

$$x \ominus_a y = x \oplus_a (-y) = (x + (a - y)) \bmod a$$

Modulare Multiplikation $\odot$:

Es gilt folgende Regel:

Inverse Element in $\odot$:

$$x \odot_a y = (x \cdot y) \bmod a$$

$$x \odot_a y = ((x \bmod a) \cdot (y \bmod a)) \bmod a$$

existiert im Allgemeinen nicht!

Wenn $\text{ggT}(x, a) = 1$ dann existiert ein inverses Element zu x

Potenz Modulo Regeln:

$$(x^{s+t}) \bmod a = ((x^s \bmod a) \cdot (x^t \bmod a)) \bmod a$$

$$(x^{s \cdot t}) \bmod a = (x^s \bmod a)^t \bmod a$$

Vigenère-Chiffre (mathematisch)



Polyalphabetisches
Substitutionsverfahren

Klartext	EINKLEINESGEHEIMNIS	(Zeile)
Schlüssel	KEYKEYKEYKEYKEYKEYK	(Spalte)
Kryptotext	OMLUPCSRCKCRIGWRGC	(Kreuzung)

		Schlüsselbuchstabe																									
		A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Klartextbuchstabe	A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
	C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
	D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
	E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
	F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
	G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
	H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
	I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
	J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
	K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
	L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
	M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
	N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
	O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
	P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
	Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
	R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
	S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
	T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
	U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
	V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
	W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V



Vigenère-Chiffre (mathematisch)

Polyalphabetisches
Substitutionsverfahren

Klartext	EINKLEINESGEHEIMNIS	(Zeile)
Schlüssel	KEYKEYKEYKEYKEYKEYK	(Spalte)
Kryptotext	OMLUPCSRCKCRIGWRGC	(Kreuzung)

	Schlüsselbuchstabe																									
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V

$$E(m_i, k_i) = (m_i + k_i) \bmod 26 = c_i$$

$$D(c_i, k_i) = (c_i - k_i) \bmod 26 = m_i$$

$$A = 0, B = 1, C = 2, \dots$$



Vigenère-Chiffre (mathematisch)

Polyalphabetisches
Substitutionsverfahren

Klartext	EINKLEINESGEHEIMNIS	(Zeile)
Schlüssel	KEYKEYKEYKEYKEYKEYK	(Spalte)
Kryptotext	OMLUPCSRCKCRIGWRGC	(Kreuzung)

Klartext	Zahl	Schlüssel	Zahl	Verschlüsselung	Geheimtext
G	6	B	1	$6 + 1 \bmod 26 = 7$	H
E	4	O	14	$4 + 14 \bmod 26 = 18$	S
H	7	N	13	$7 + 13 \bmod 26 = 20$	U
E	4	D	3	$4 + 3 \bmod 26 = 7$	H
I	8	B	1	$8 + 1 \bmod 26 = 9$	J
M	12	O	14	$12 + 14 \bmod 26 = 0$	A

Verschlüsselung

	Schlüsselbuchstabe																									
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V

Klartextbuchstabe i

Schlüsselbuchstabe i

$$E(m_i, k_i) = (m_i + k_i) \bmod 26 = c_i$$

$$D(c_i, k_i) = (c_i - k_i) \bmod 26 = m_i$$

A = 0, B = 1, C = 2, ...

Kryptobuchstabe i

<https://youtu.be/N31r393rEUk>

K
A

- **Data Encryption Standard (DES)** weit verbreiteter **symmetrischer Verschlüsselungsalgorithmus**
- offizieller Standard für die US-Regierung (1977)
- heutzutage aufgrund der verwendeten Schlüssellänge (56 Bits) nicht ausreichend sicher
- Durch Mehrfachanwendung als Triple-DES weiterhin am häufigsten, z.B. von Banken in Chipkartenanwendungen, eingesetzt



- Data Encryption Standard (DES) weit verbreiteter **symmetrischer Verschlüsselungsalgorithmus**
- offizieller Standard für die US-Regierung (1977)
- heutzutage aufgrund der verwendeten Schlüssellänge (56 Bits) nicht ausreichend sicher
- Durch Mehrfachanwendung als Triple-DES weiterhin am häufigsten, z.B. von Banken in Chipkartenanwendungen, eingesetzt
- **Neuer offizieller Standard** für die USA: **Advanced Encryption Standard (AES)** seit 2000



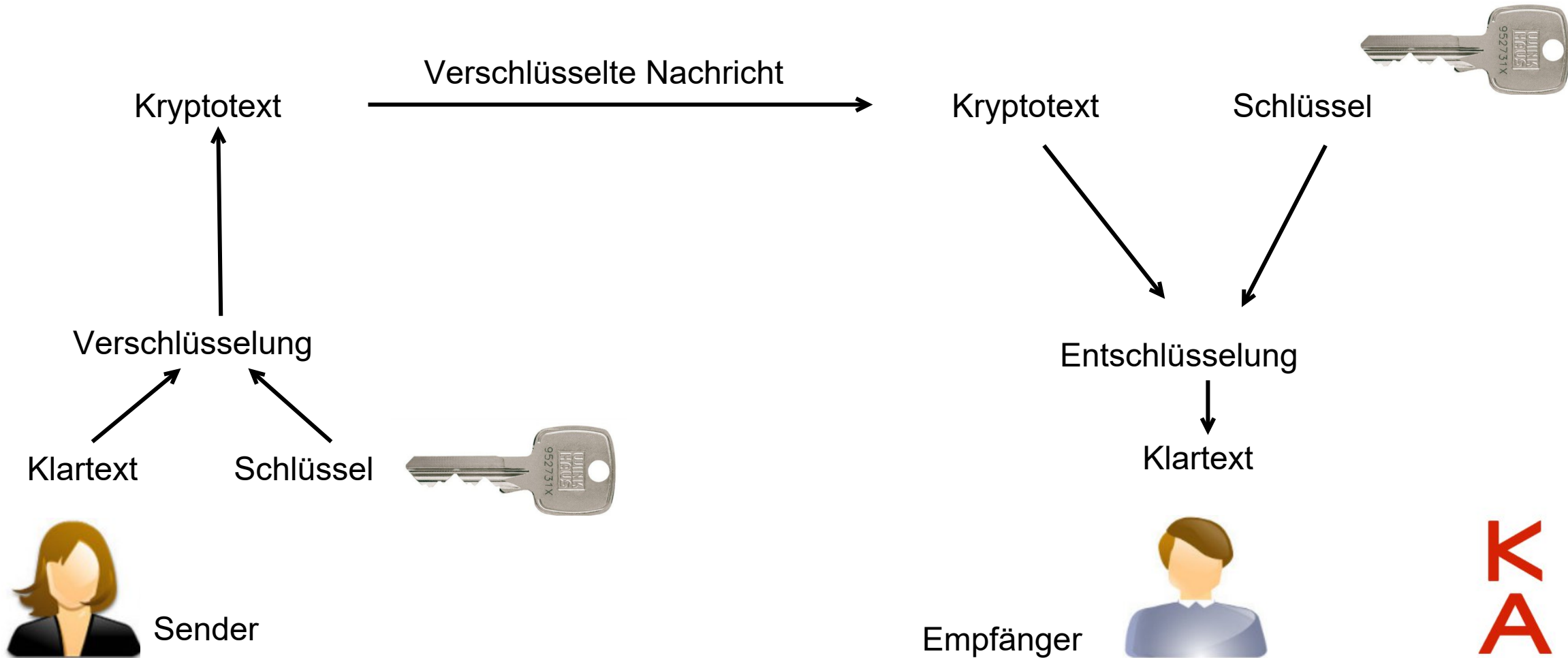
- Data Encryption Standard (DES) weit verbreiteter symmetrischer Verschlüsselungsalgorithmus
- offizieller Standard für die US-Regierung (1977)
- heutzutage aufgrund der verwendeten Schlüssellänge (56 Bits) nicht ausreichend sicher
- Durch Mehrfachanwendung als Triple-DES weiterhin am häufigsten, z.B. von Banken in Chipkartenanwendungen, eingesetzt
- Neuer offizieller Standard für die USA: Advanced Encryption Standard (AES) seit 2000

DES & AES verwenden:

- Blockverschlüsselung
(der Klartext wird in Blöcke gleicher Länge geteilt und einzeln verschlüsselt)
- Geschickte Kombination von drei einfachen Grundoptionen:
 1. Ersetzungen*
(Teilblöcke werden durch Teilblöcke nach Tabellen ersetzt; vergl. z.B. CAESAR, VIGENÉRE)
 2. Vertauschungen
(Austauschen einzelner Teilblöcke eines Blocks untereinander; vergl. SKYTALE)
 3. Maskierung
(Teilblöcke werden modular zu Teilschlüsseln addiert)

*„Diese nicht-lineare Transformierung in den S-Boxen stellt das Herzstück der Sicherheit von DES dar, ohne sie wäre DES linear und trivial zu brechen.“

Symmetrische Verschlüsselungsverfahren

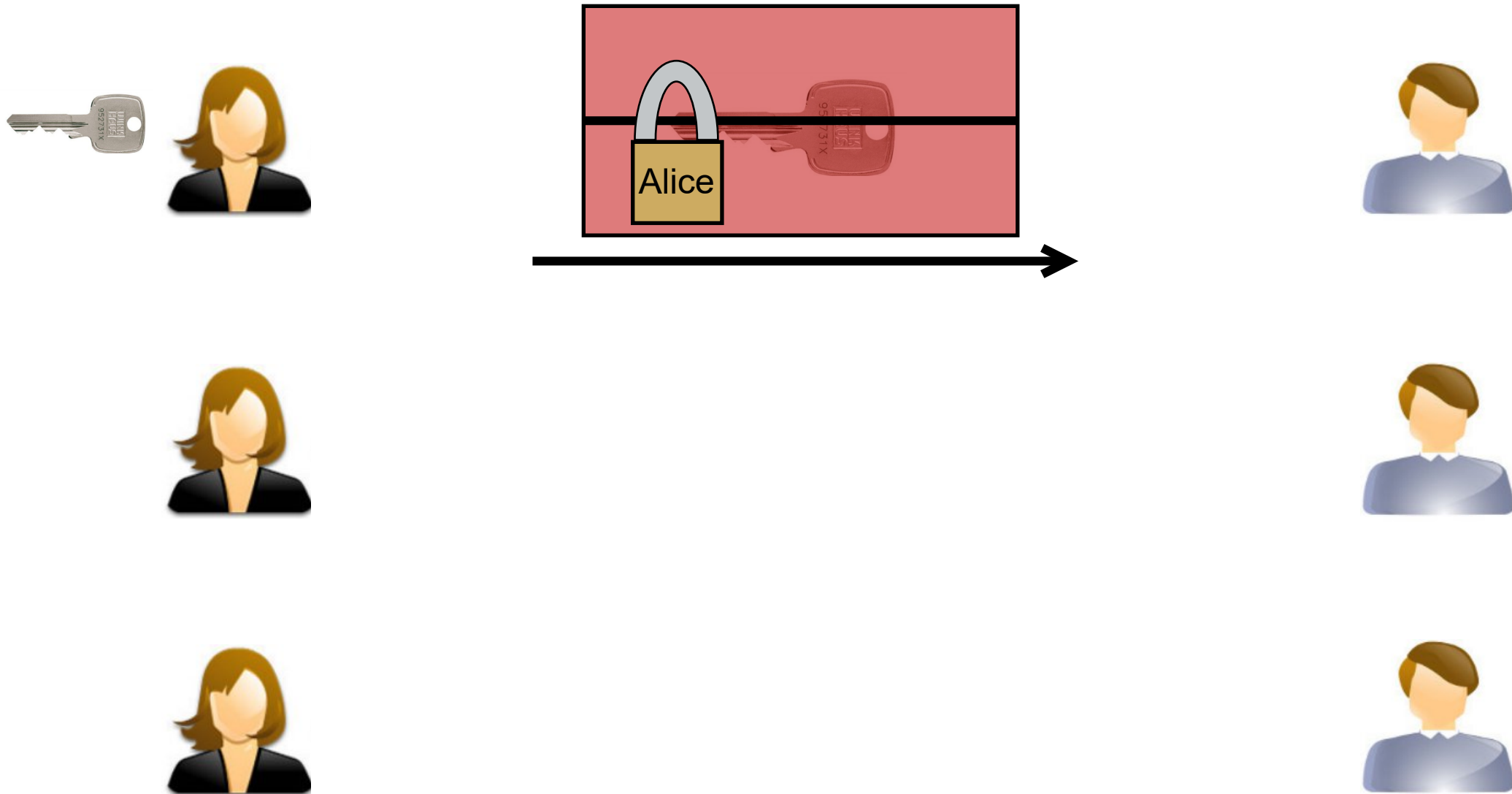


Schlüsselaustausch?



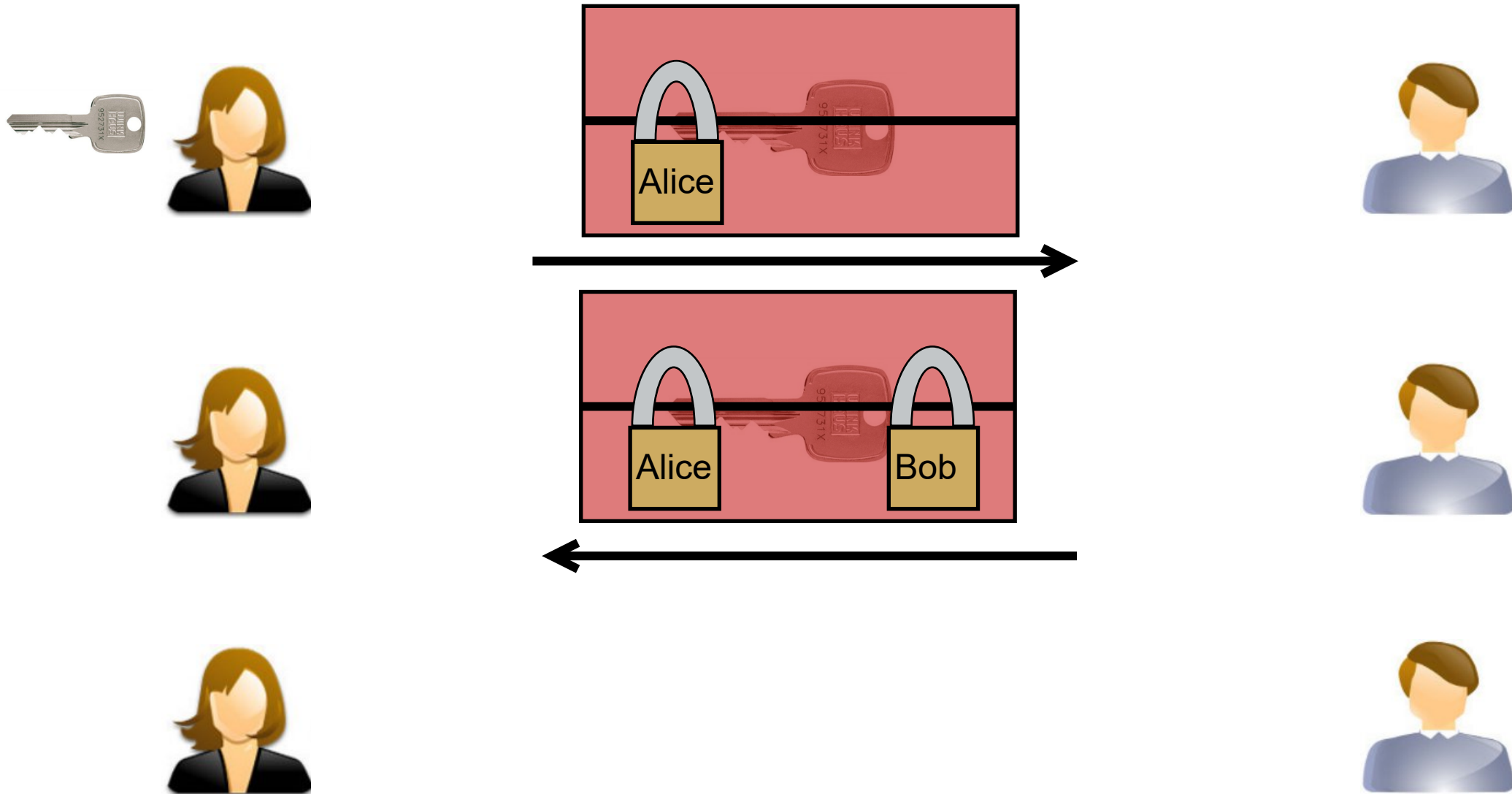
K
A

Schlüsselaustausch?



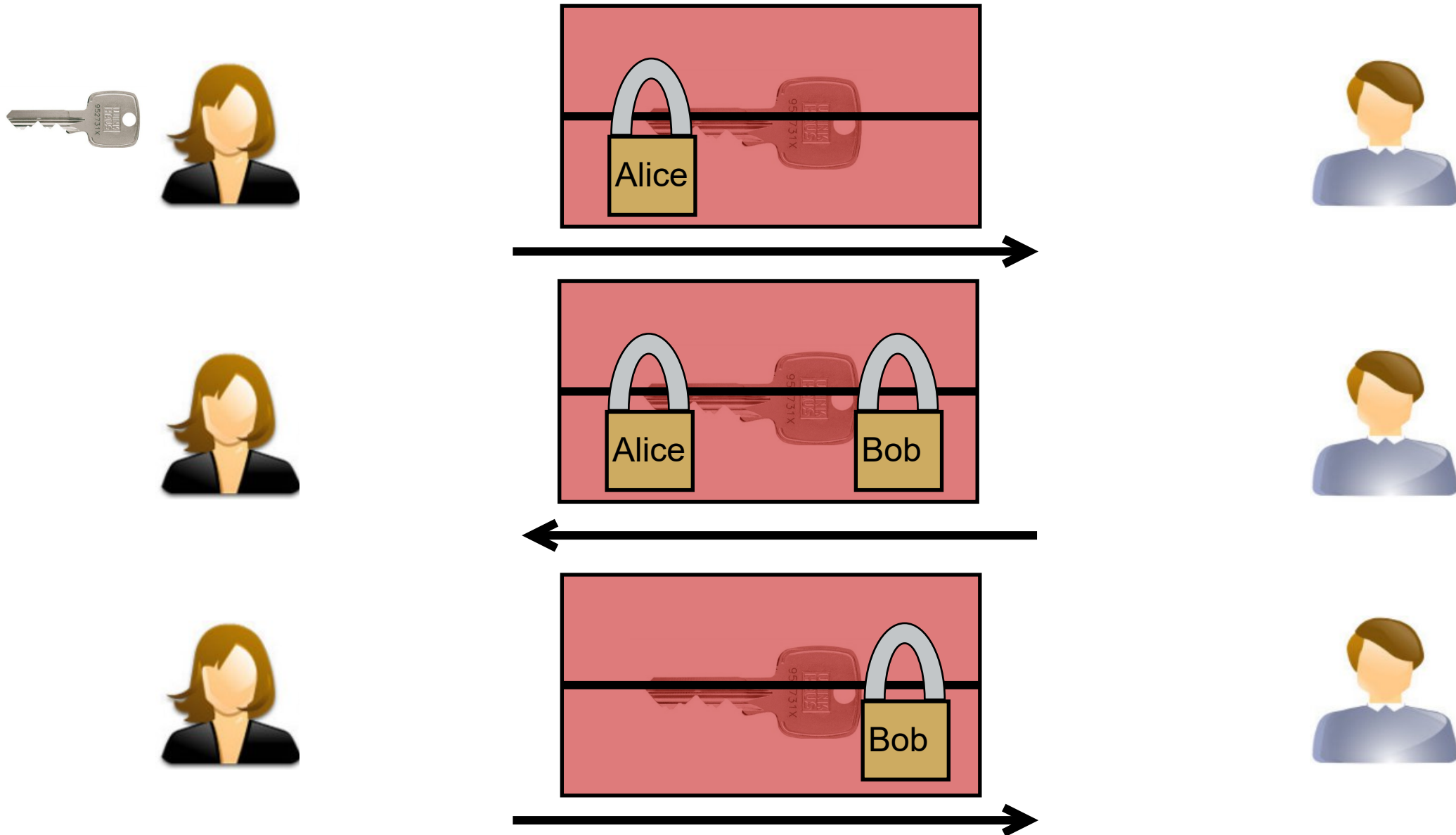
K
A

Schlüsselaustausch?



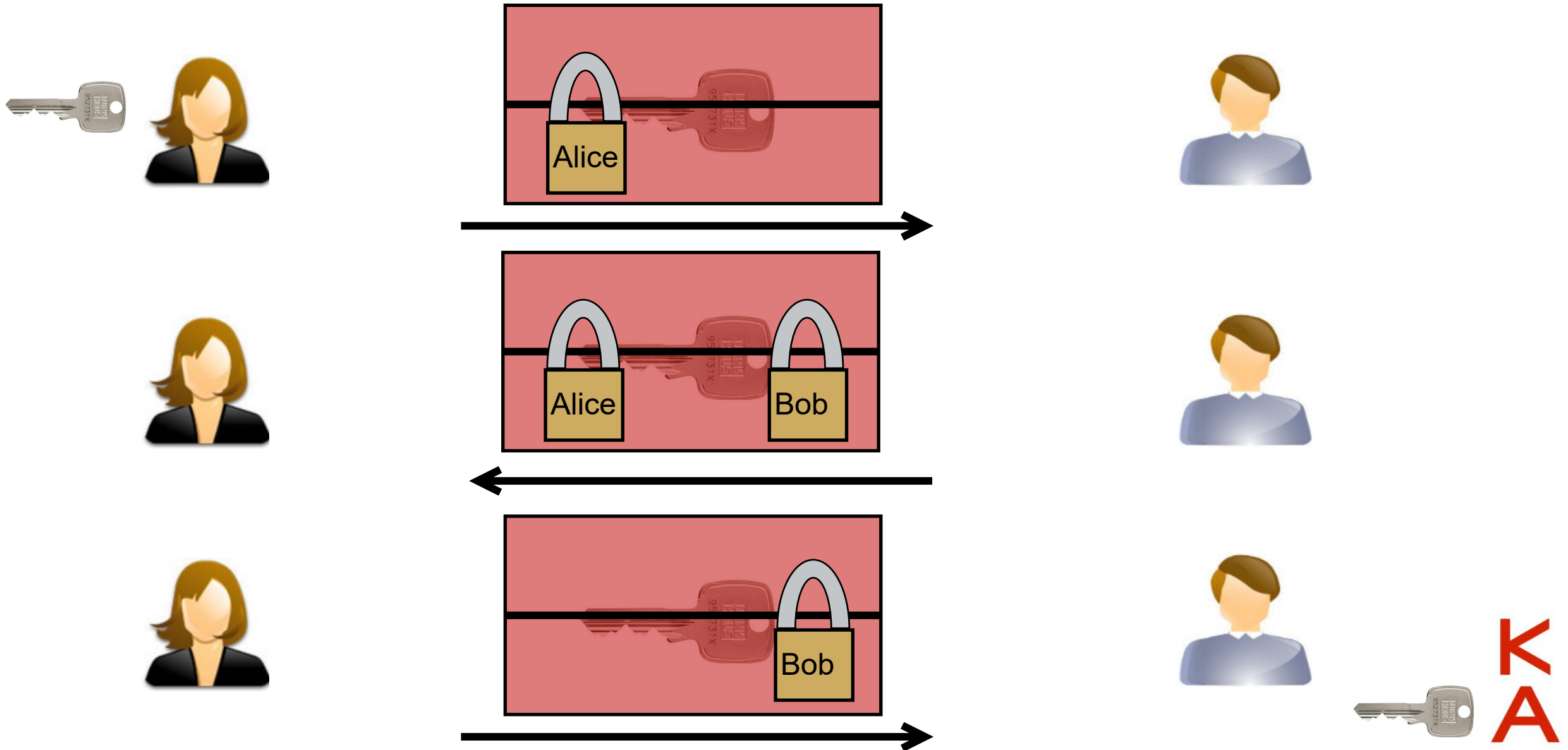
K
A

Schlüsselaustausch?



K
A

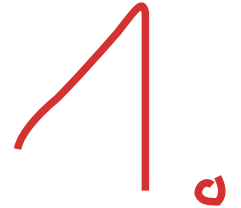
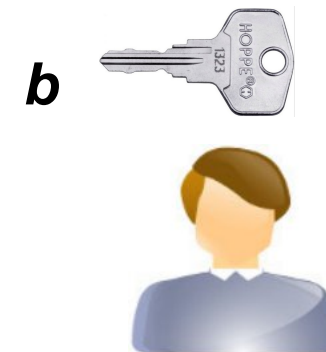
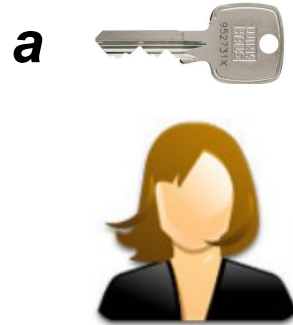
Schlüsselaustausch!



Statt Schlüsselaustausch: Berechnen eines gemeinsamen Schlüssels 

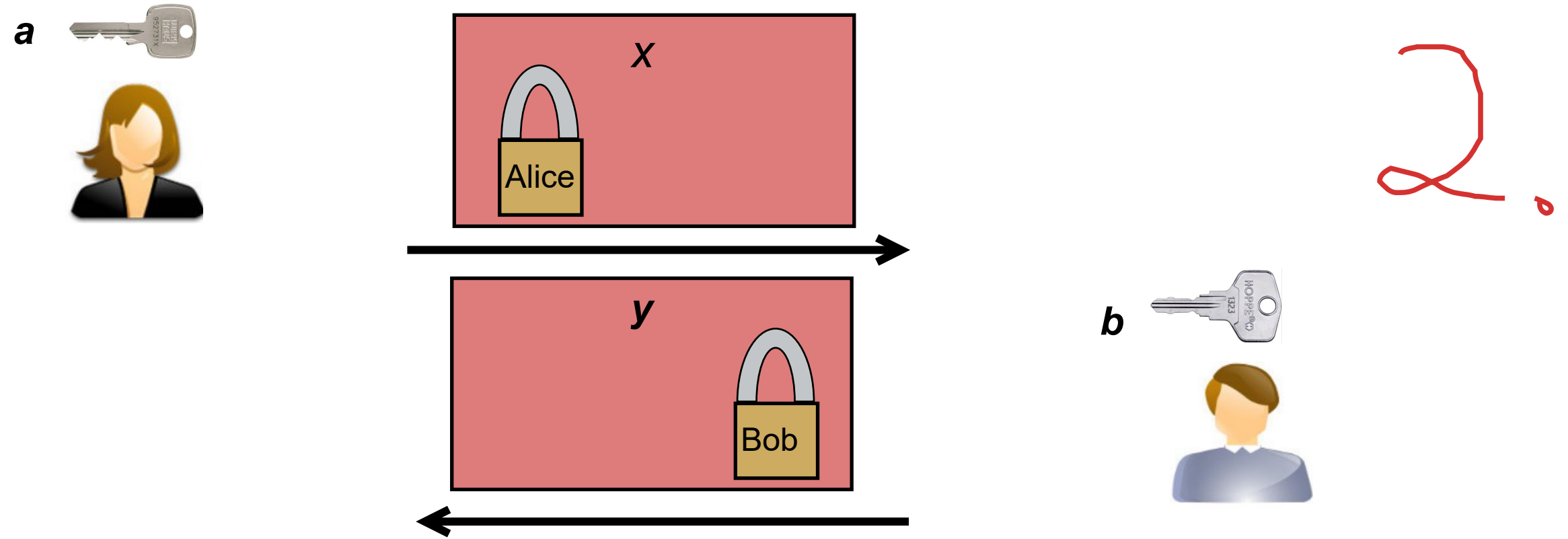
Diffie-Hellman Protokoll

Diffie-Hellman Protokoll: Berechnen eines gemeinsamen Schlüssels **+**



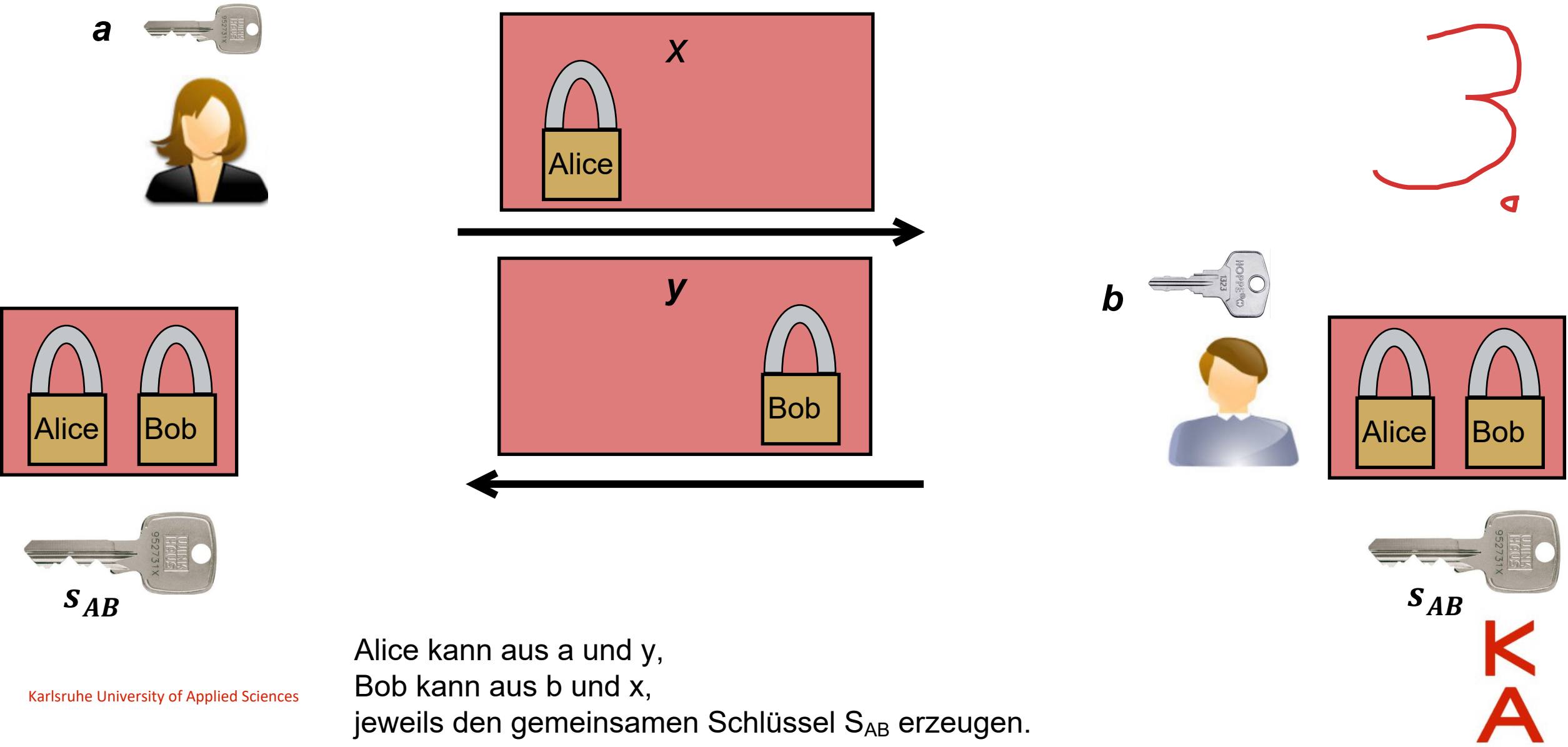
Alice und Bob haben jeweils geheime Schlüssel a bzw. b.

Diffie-Hellman Protokoll: Berechnen eines gemeinsamen Schlüssels **+**



Alice und Bob schicken sich gegenseitig mit den geheime Schlüssel a bzw. b erzeugte Botschaften x bzw. y.

Diffie-Hellman Protokoll: Berechnen eines gemeinsamen Schlüssels **+**



Diffie-Hellman Protokoll



Alice & Bob einigen sich auf zwei öffentliche natürliche Zahlen c und p . Dabei ist p eine Primzahl und $c < p$.



Diffie-Hellman Protokoll

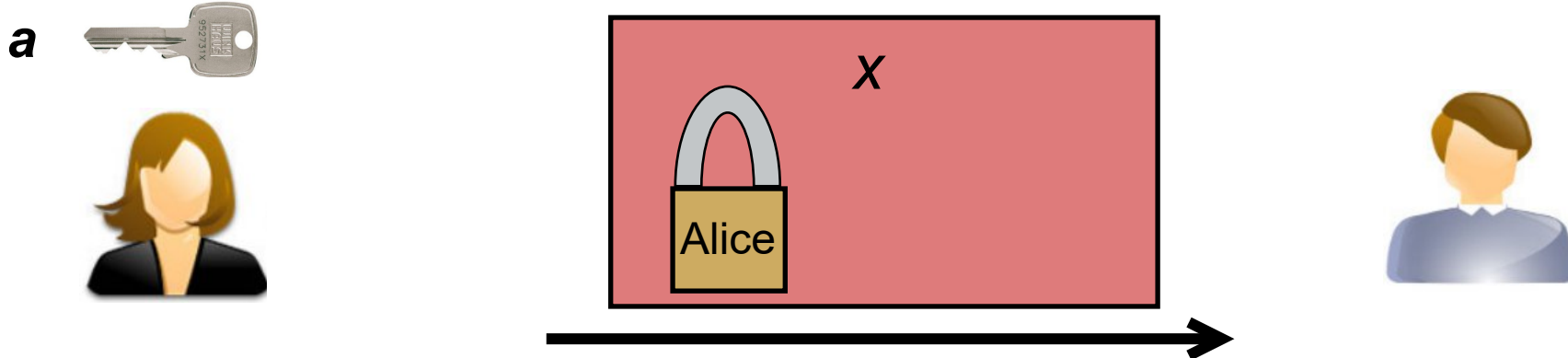


Alice & Bob einigen sich auf zwei öffentliche natürliche Zahlen c und p . Dabei ist p eine Primzahl und $c < p$.

1. Alice wählt eine zufällige Zahl $a < p$, hält diese geheim und berechnet die Zahl

$$x = c^a \bmod p$$

Sie schickt x an Bob.



Diffie-Hellman Protokoll



Alice & Bob einigen sich auf zwei öffentliche natürliche Zahlen c und p . Dabei ist p eine Primzahl und $c < p$.

1. Alice wählt eine zufällige Zahl $a < p$, hält diese geheim und berechnet die Zahl

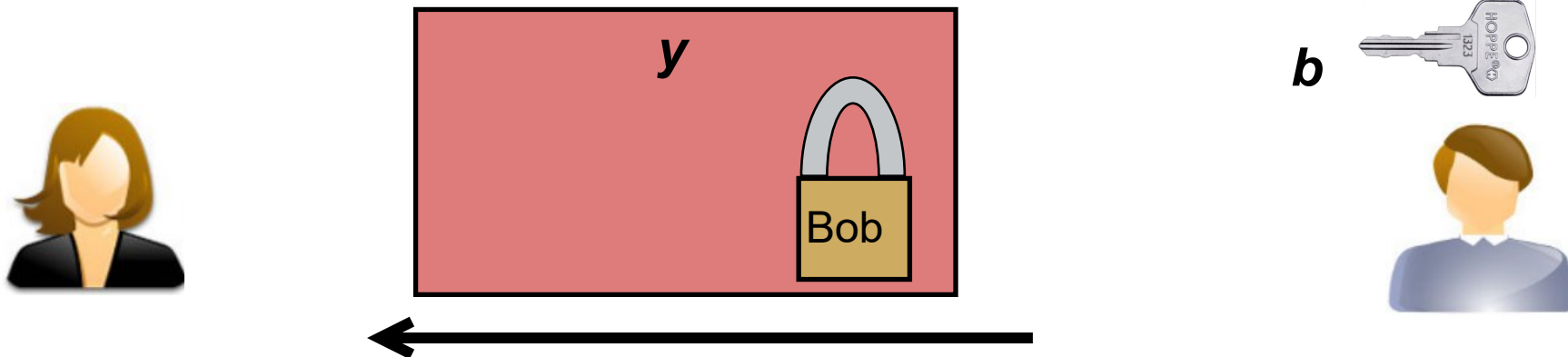
$$x = c^a \bmod p$$

Sie schickt x an Bob.

2. Bob wählt eine zufällige Zahl b , hält diese geheim und berechnet die Zahl

$$y = c^b \bmod p$$

Er schickt y an Alice.



Diffie-Hellman Protokoll



Alice & Bob einigen sich auf zwei öffentliche natürliche Zahlen c und p . Dabei ist p eine Primzahl und $c < p$.

1. Alice wählt eine zufällige Zahl $a < p$, hält diese geheim und berechnet die Zahl

$$x = c^a \bmod p$$

Sie schickt x an Bob.

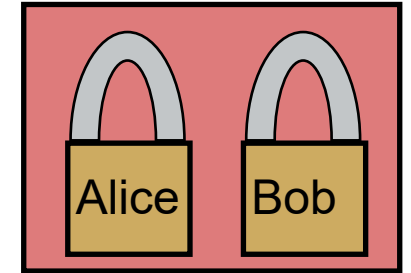
2. Bob wählt eine zufällige Zahl b , hält diese geheim und berechnet die Zahl

$$y = c^b \bmod p$$

Er schickt y an Alice.

3. Alice erhält y von Bob und berechnet mit ihrer geheimen Zahl a den Schlüssel

$$s_{AB} = y^a \bmod p$$



s_{AB}

Diffie-Hellman Protokoll



Alice & Bob einigen sich auf zwei öffentliche natürliche Zahlen c und p . Dabei ist p eine Primzahl und $c < p$.

1. Alice wählt eine zufällige Zahl $a < p$, hält diese geheim und berechnet die Zahl

$$x = c^a \bmod p$$

Sie schickt x an Bob.

2. Bob wählt eine zufällige Zahl b , hält diese geheim und berechnet die Zahl

$$y = c^b \bmod p$$

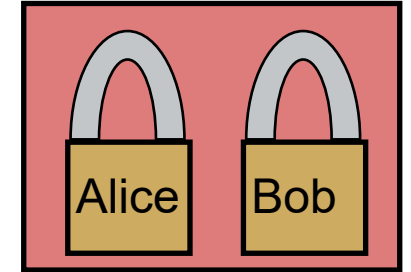
Er schickt y an Alice.

3. Alice erhält y von Bob und berechnet mit ihrer geheimen Zahl a den Schlüssel

$$s_{AB} = y^a \bmod p$$

4. Bob erhält x von Alice und berechnet mit seiner geheimen Zahl b den Schlüssel

$$s_{BA} = x^b \bmod p$$



s_{AB}



s_{BA}



Diffie-Hellman Protokoll



Alice & Bob einigen sich auf zwei öffentliche natürliche Zahlen c und p . Dabei ist p eine Primzahl und $c < p$.

1. Alice wählt eine zufällige Zahl $a < p$, hält diese geheim und berechnet die Zahl

$$x = c^a \bmod p$$

Sie schickt x an Bob.

2. Bob wählt eine zufällige Zahl b , hält diese geheim und berechnet die Zahl

$$y = c^b \bmod p$$

Er schickt y an Alice.

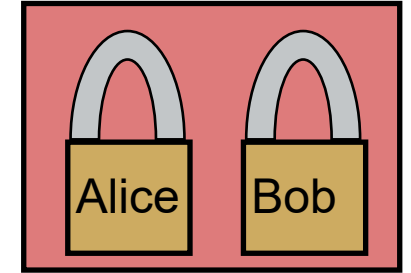
3. Alice erhält y von Bob und berechnet mit ihrer geheimen Zahl a den Schlüssel

$$s_{AB} = y^a \bmod p$$

4. Bob erhält x von Alice und berechnet mit seiner geheimen Zahl b den Schlüssel

$$s_{BA} = x^b \bmod p$$

Da $s_{AB} = s_{BA}$ gilt, haben beide die **gleiche Zahl** berechnet. Diese Zahl ist der gemeinsame Schlüssel.



s_{AB}



s_{BA}

Diffie-Hellman Protokoll (Beispiel)



Alice & Bob einigen sich auf zwei öffentliche natürliche Zahlen $c = 3$ und $p = 17$.
Dabei ist p eine Primzahl und $c < p$.

1. Alice wählt eine zufällige Zahl $a = 6 < p$, hält diese geheim und berechnet die Zahl
$$x = c^a \bmod p = 3^6 \bmod 17 = 729 \bmod 17 = 15$$

Sie schickt $x = 15$ an Bob.

Diffie-Hellman Protokoll (Beispiel)

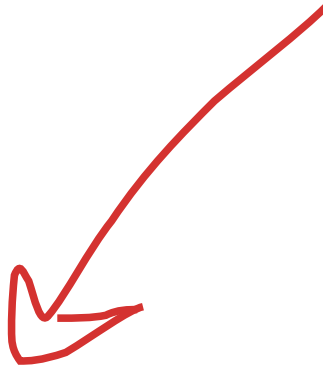


Alice & Bob einigen sich auf zwei öffentliche natürliche Zahlen $c = 3$ und $p = 17$.
Dabei ist p eine Primzahl und $c < p$.

1. Alice wählt eine zufällige Zahl $a = 6 < p$, hält diese geheim und berechnet die Zahl

$$x = c^a \bmod p = 3^6 \bmod 17 = 729 \bmod 17 = 15$$

Sie schickt $x = 15$ an Bob.



$$3^6 \bmod 17 = 3^{3 \cdot 2} \bmod 17 = (3^3)^2 \bmod 17 = (\underbrace{3^3}_{x'} \cdot \underbrace{3^3}_{y'}) \bmod 17$$

Diffie-Hellman Protokoll (Beispiel)

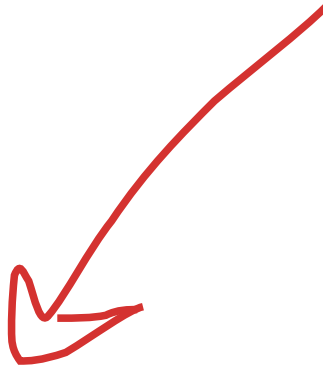


Alice & Bob einigen sich auf zwei öffentliche natürliche Zahlen $c = 3$ und $p = 17$.
Dabei ist p eine Primzahl und $c < p$.

1. Alice wählt eine zufällige Zahl $a = 6 < p$, hält diese geheim und berechnet die Zahl

$$x = c^a \bmod p = 3^6 \bmod 17 = 729 \bmod 17 = 15$$

Sie schickt $x = 15$ an Bob.



$$3^6 \bmod 17 = 3^{3 \cdot 2} \bmod 17 = (3^3)^2 \bmod 17 = (\underbrace{3^3}_{x'} \cdot \underbrace{3^3}_{y'}) \bmod 17 = (\underbrace{(3^3 \bmod 17)(3^3 \bmod 17)}_{(3^3 \bmod 17)^2}) \bmod 17$$

Reminder: Potenz-Modulo Rechnen



Den Rest großer Potenzen mit „Modularem Potenzieren“ berechnen

Beim Potenzieren von zwei Zahlen entstehen schnell große Zahlen, mit denen das Rechnen mühsam oder, wenn die Anzeige des Taschenrechners sie nicht mehr vollständig anzeigen lässt, unmöglich wird. Soll jedoch der **Rest** einer solchen Potenz berechnet werden kann man die Rechnung mit Hilfe von zwei Tricks vereinfachen:

(Regel I) Lässt sich der Exponent als **Summe** zweier kleinerer Zahlen darstellen, so gilt:

$$x^{a+b} = x^a \cdot x^b$$

Soll der Rest der Potenz gebildet werden, so kann die Restfunktion bereits auf Zwischenergebnisse angewendet werden, denn bezüglich der Restbildung zur Division durch d gilt:

$$x^{a+b} \bmod d = ((x^a \bmod d) \cdot (x^b \bmod d)) \bmod d$$

(Regel II) Lässt sich der Exponent als **Produkt** zweier kleinerer Zahlen darstellen, so gilt:

$$x^{a \cdot b} = (x^a)^b$$

Soll der Rest der Potenz gebildet werden, so kann die Restfunktion bereits auf Zwischenergebnisse angewendet werden, denn bezüglich der Restbildung zur Division durch d gilt:

$$x^{a \cdot b} \bmod d = (x^a \bmod d)^b \bmod d$$



Diffie-Hellman Protokoll (Beispiel)



Alice & Bob einigen sich auf zwei öffentliche natürliche Zahlen $c = 3$ und $p = 17$.
Dabei ist p eine Primzahl und $c < p$.

1. Alice wählt eine zufällige Zahl $a = 6 < p$, hält diese geheim und berechnet die Zahl
$$x = c^a \bmod p = 3^6 \bmod 17 = 729 \bmod 17 = 15$$

Sie schickt $x = 15$ an Bob.

2. Bob wählt eine zufällige Zahl $b = 4$, hält diese geheim und berechnet die Zahl
$$y = c^b \bmod p = 3^4 \bmod 17 = 81 \bmod 17 = 13$$

Er schickt $y = 13$ an Alice.

Diffie-Hellman Protokoll (Beispiel)



Alice & Bob einigen sich auf zwei öffentliche natürliche Zahlen $c = 3$ und $p = 17$.
Dabei ist p eine Primzahl und $c < p$.

1. Alice wählt eine zufällige Zahl $a = 6 < p$, hält diese geheim und berechnet die Zahl
$$x = c^a \bmod p = 3^6 \bmod 17 = 729 \bmod 17 = 15$$

Sie schickt $x = 15$ an Bob.

2. Bob wählt eine zufällige Zahl $b = 4$, hält diese geheim und berechnet die Zahl
$$y = c^b \bmod p = 3^4 \bmod 17 = 81 \bmod 17 = 13$$

Er schickt $y = 13$ an Alice.

3. Alice erhält $y = 13$ von Bob und berechnet mit ihrer geheimen Zahl a den Schlüssel

$$s_{AB} = y^a \bmod p = 13^6 \bmod 17 = (13^2)^3 \bmod 17 = (13^2 \bmod 17)^3 \bmod 17 = 16^3 \bmod 17 = 16$$



Diffie-Hellman Protokoll (Beispiel)



Alice & Bob einigen sich auf zwei öffentliche natürliche Zahlen $c = 3$ und $p = 17$.
Dabei ist p eine Primzahl und $c < p$.

1. Alice wählt eine zufällige Zahl $a = 6 < p$, hält diese geheim und berechnet die Zahl

$$x = c^a \bmod p = 3^6 \bmod 17 = 729 \bmod 17 = 15$$

Sie schickt $x = 15$ an Bob.

2. Bob wählt eine zufällige Zahl $b = 4$, hält diese geheim und berechnet die Zahl

$$y = c^b \bmod p = 3^4 \bmod 17 = 81 \bmod 17 = 13$$

Er schickt $y = 13$ an Alice.

3. Alice erhält $y = 13$ von Bob und berechnet mit ihrer geheimen Zahl a den Schlüssel

$$s_{AB} = y^a \bmod p = 13^6 \bmod 17 = (13^2)^3 \bmod 17 = (13^2 \bmod 17)^3 \bmod 17 = 16^3 \bmod 17 = 16$$

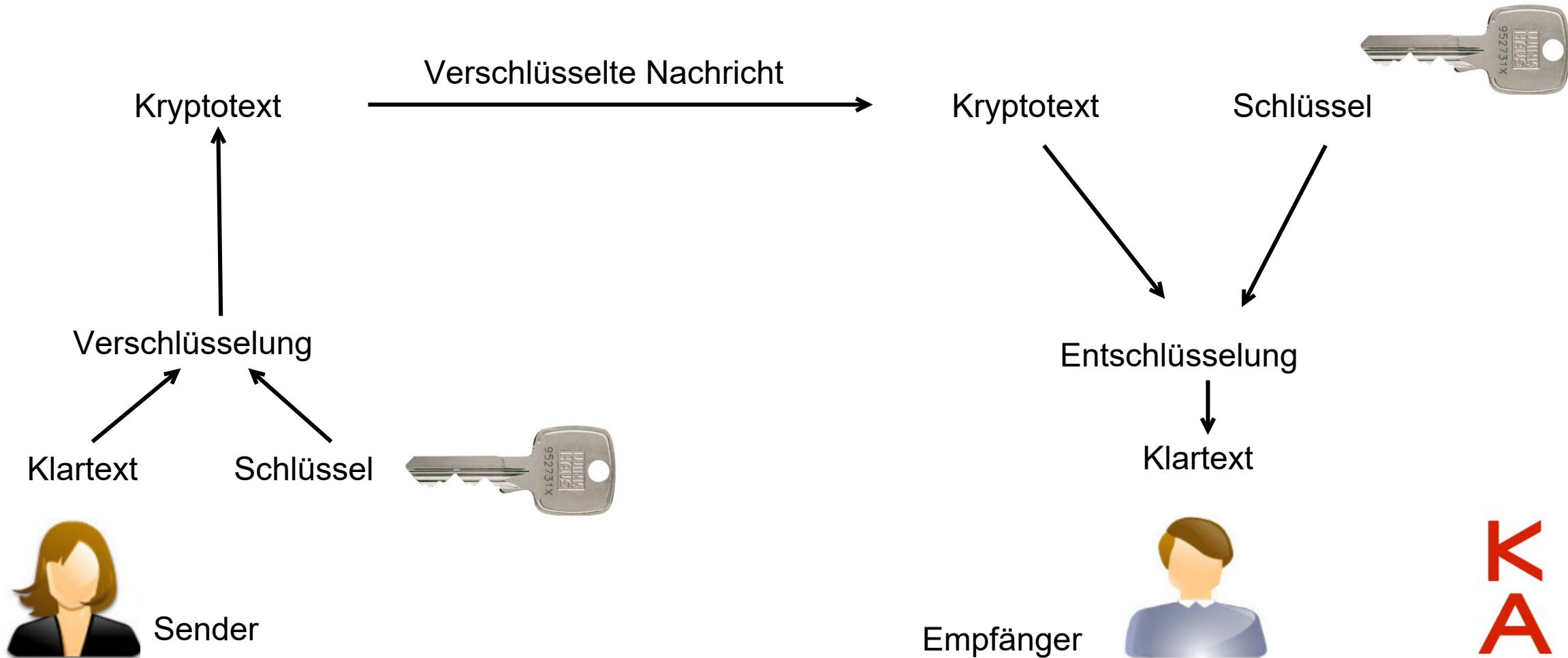
4. Bob erhält $x = 15$ von Alice und berechnet mit seiner geheimen Zahl b den Schlüssel

$$s_{BA} = x^b \bmod p = 15^4 \bmod 17 = (15^2 \bmod 17)^2 \bmod 17 = 16$$

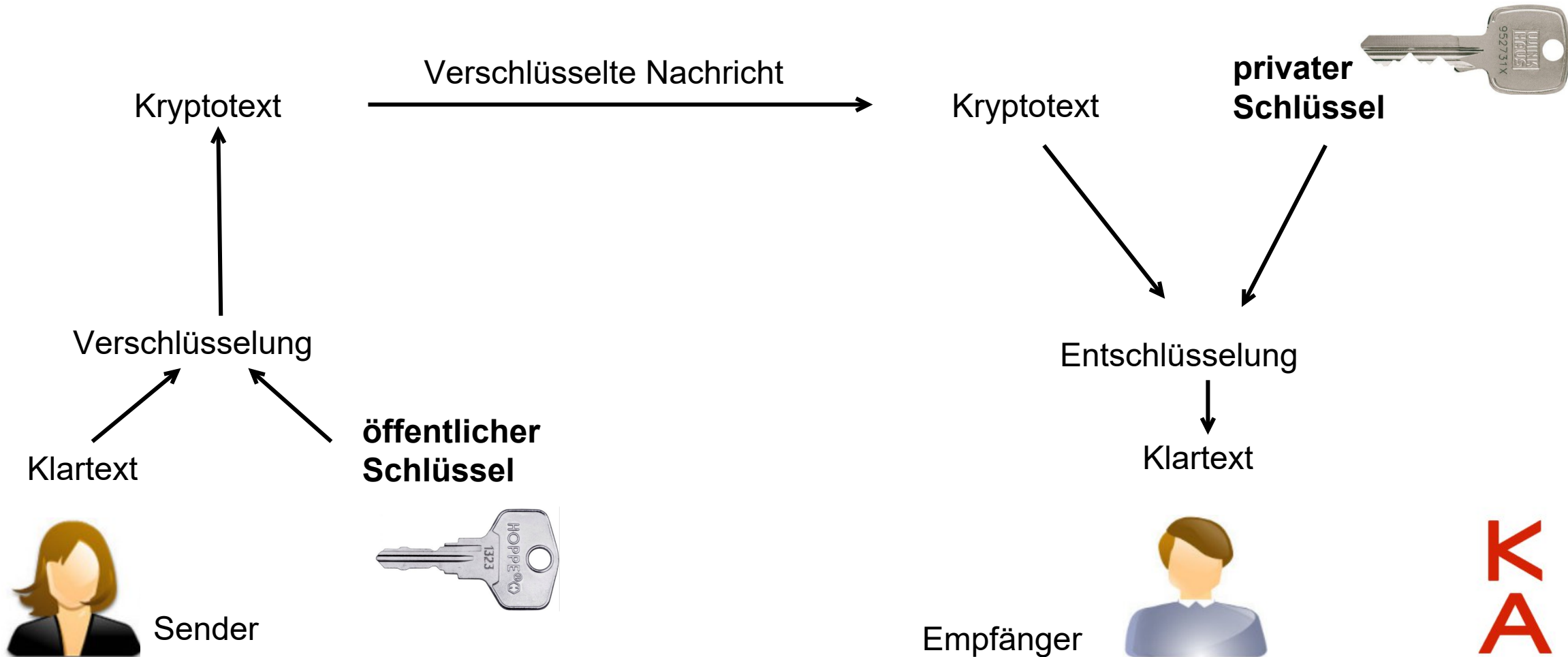
Da $s_{AB} = s_{BA} = 16$ gilt, haben beide die gleiche Zahl berechnet. Diese Zahl ist der gemeinsame Schlüssel.



Bisher: Symmetrische Verschlüsselungsverfahren

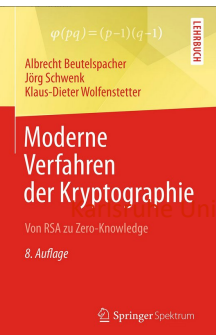


Asymmetrische Verschlüsselungsverfahren



RSA (Rivest–Shamir–Adleman) ist ein asymmetrisches kryptographisches Verfahren, das sowohl zum Verschlüsseln als auch zum digitalen Signieren verwendet werden kann.

Ronald L. Rivest, Adi Shamir und Leonard Adleman versuchten 1978 am MIT die Theorie von Diffie und Hellman zur Public-Key-Kryptografie zu widerlegen. Allerdings fanden sie ein Verfahren ohne Angriffspunkte.



Der Empfänger Bob

1. wählt zufällige große Primzahlen p und q . Daraus berechnet er

$$n = p \cdot q$$

Der Empfänger Bob

1. wählt zufällige große Primzahlen p und q . Daraus berechnet er

$$n = p \cdot q$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1)$$

Der Empfänger Bob

1. wählt zufällige große Primzahlen p und q . Daraus berechnet er

$$n = p \cdot q$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1)$$

3. wählt zufällig eine große Zahl $e \in Z_n$ mit $ggT(e, \phi(n)) = 1$

Der Empfänger Bob

1. wählt zufällige große Primzahlen p und q . Daraus berechnet er

$$n = p \cdot q$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1)$$

3. wählt zufällig eine große Zahl $e \in Z_n$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h.

$$(d \cdot e) \bmod \phi(n) = 1$$

Der Empfänger Bob

1. wählt zufällige große Primzahlen p und q . Daraus berechnet er

$$n = p \cdot q$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1)$$

3. wählt zufällig eine große Zahl $e \in Z_n$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h.

$$(d \cdot e) \bmod \phi(n) = 1$$

Der öffentliche Schlüssel ist (n, e) und

der private Schlüssel ist $(p, q, \phi(n), d)$. Wobei $(p, q, \phi(n))$ nicht mehr benötigt und vernichtet werden können.

Verschlüsselung: $f_e(m) = m^e \bmod n = c$

Entschlüsselung: $f_d(c) = c^d \bmod n$

Public-Key Eigenschaft:

Wenn man nur die Zahl n kennt (und nicht etwa die Faktoren p und q oder die Zahl $\phi(n) = (p - 1)(q - 1)$), so kann man aus e nicht d berechnen.

RSA (Beispiel)



Der Empfänger Bob

1. wählt zufällige große Primzahlen $p = 11$ und $q = 17$. Daraus berechnet er

$$n = p \cdot q = 11 \cdot 17 = 187$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1) = 10 \cdot 16 = 160$$

3. wählt zufällig eine große Zahl $e \in Z_n$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h.

$$(d \cdot e) \bmod \phi(n) = 1$$

RSA (Beispiel)



Der Empfänger Bob

1. wählt zufällige große Primzahlen $p = 11$ und $q = 17$. Daraus berechnet er

$$n = p \cdot q = 11 \cdot 17 = 187$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1) = 10 \cdot 16 = 160$$

3. wählt zufällig eine große Zahl $e \in Z_n$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h

$$(d \cdot e) \bmod \phi(n) = 1$$



$$\phi(n) = 160 = 2^5 \cdot 5 \quad (\text{Primfaktorzerlegung})$$

$$ggT(e, \phi(n)) = 1 \quad (e = 3, 7, 9, 11 \text{ erfüllen die Bedingung})$$

RSA (Beispiel)



Der Empfänger Bob

1. wählt zufällige große Primzahlen $p = 11$ und $q = 17$. Daraus berechnet er

$$n = p \cdot q = 11 \cdot 17 = 187$$

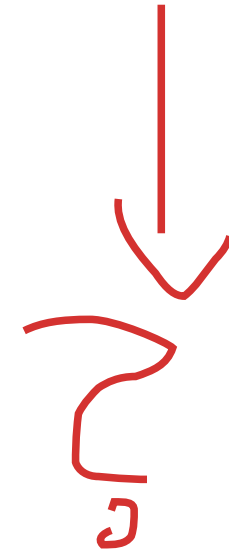
2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1) = 10 \cdot 16 = 160$$

3. wählt zufällig eine große Zahl $e = 7$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h.
 $(d \cdot e) \bmod \phi(n) = 1$



$$\begin{aligned} \phi(n) &= 160 = 2^5 \cdot 5 \quad (\text{Primfaktorzerlegung}) \\ ggT(e, \phi(n)) &= 1 \quad (e = 3, 7, 9, 11 \text{ erfüllen die Bedingung}) \end{aligned}$$



RSA (Beispiel)



Der Empfänger Bob

1. wählt zufällige große Primzahlen $p = 11$ und $q = 17$. Daraus berechnet er

$$n = p \cdot q = 11 \cdot 17 = 187$$

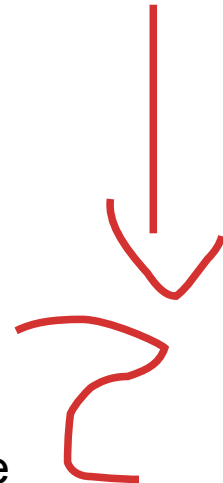
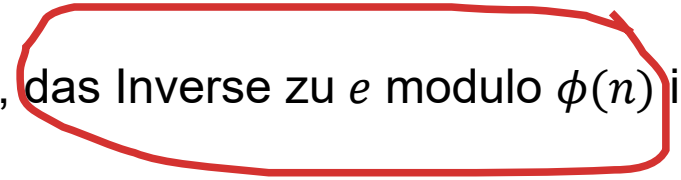
2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1) = 10 \cdot 16 = 160$$

3. wählt zufällig eine große Zahl $e = 7$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h.
 $(d \cdot e) \bmod \phi(n) = 1$



$$\begin{aligned} \phi(n) = 160 &= 2^5 \cdot 5 \quad (\text{Primfaktorzerlegung}) \\ ggT(e, \phi(n)) &= 1 \quad (e = 3, 7, 9, 11 \text{ erfüllen die Bedingung}) \end{aligned}$$



Theoretisch möglich alle
auszuprobieren... Praktisch
nicht...

RSA (Beispiel)



Der Empfänger Bob

1. wählt zufällige große Primzahlen $p = 11$ und $q = 17$. Daraus berechnet er

$$n = p \cdot q = 11 \cdot 17 = 187$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1) = 10 \cdot 16 = 160$$

3. wählt zufällig eine große Zahl $e = 7$ mit $ggT(e, \phi(n)) = 1$ und berechnet $d = 23$, das Inverse zu e modulo $\phi(n)$, d.h.
 $(d \cdot e) \bmod \phi(n) = 1$



$$\begin{aligned} \phi(n) = 160 &= 2^5 \cdot 5 \quad (\text{Primfaktorzerlegung}) \\ ggT(e, \phi(n)) &= 1 \quad (e = 3, 7, 9, 11 \text{ erfüllen die Bedingung}) \end{aligned}$$



Der erweiterten Euklidischen Algorithmus* liefert $d = 23$.

$$(7 \cdot 23) \bmod 160 = 1$$

RSA (Beispiel)



Der Empfänger Bob

1. wählt zufällige große Primzahlen $p = 11$ und $q = 17$. Daraus berechnet er

$$n = p \cdot q = 11 \cdot 17 = 187$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1) = 10 \cdot 16 = 160$$

3. wählt zufällig eine große Zahl $e = 7$ mit $ggT(e, \phi(n)) = 1$ und berechnet $d = 23$, das Inverse zu e modulo $\phi(n)$, d.h.

$$(d \cdot e) \bmod \phi(n) = 1$$

Klartext $m=13$

Verschlüsselung: $f_e(m) = m^e \bmod n = 13^7 \bmod 187 = 106$

Entschlüsselung: $f_d(c) = c^d \bmod n = 106^{23} \bmod 187 = 13$

Wieso ist RSA sicher?



Der Empfänger Bob

1. wählt zufällige große Primzahlen p und q . Daraus berechnet er

$$n = p \cdot q$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1)$$

3. wählt zufällig eine große Zahl $e \in Z_n$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h.

$$(d \cdot e) \bmod \phi(n) = 1$$

Der öffentliche Schlüssel ist (n, e) und

der private Schlüssel ist $(p, q, \phi(n), d)$. Wobei $(p, q, \phi(n))$ nicht mehr benötigt und vernichtet werden können.

Verschlüsselung: $f_e(m) = m^e \bmod n = c$

Entschlüsselung: $f_d(c) = c^d \bmod n$

Public-Key Eigenschaft:

Wenn man nur die Zahl n kennt (und nicht etwa die Faktoren p und q oder die Zahl $\phi(x) = (p - 1)(q - 1)$), so kann man aus e nicht d berechnen.



Wieso ist RSA sicher?



Der Empfänger Bob

1. wählt zufällige große Primzahlen p und q . Daraus berechnet er

$$n = p \cdot q$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1)$$

3. wählt zufällig eine große Zahl $e \in Z_n$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h.

$$(d \cdot e) \bmod \phi(n) = 1$$

Der öffentliche Schlüssel ist (n, e) und

der private Schlüssel ist $(p, q, \phi(n), d)$. Wobei $(p, q, \phi(n))$ nicht mehr benötigt und vernichtet werden können.

Verschlüsselung: $f_e(m) = m^e \bmod n = c$

Entschlüsselung: $f_d(c) = c^d \bmod n$



1. Ohne Kenntnis der Primfaktoren p und q kann $\phi(n)$ nicht berechnet werden!

Public-Key Eigenschaft:

Wenn man nur die Zahl n kennt (und nicht etwa die Faktoren p und q oder die Zahl $\phi(x) = (p - 1)(q - 1)$), so kann man aus e nicht d berechnen.

K
A

Wieso ist RSA sicher?



Der Empfänger Bob

1. wählt zufällige große Primzahlen p und q . Daraus berechnet er

$$n = p \cdot q$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1)$$

3. wählt zufällig eine große Zahl $e \in Z_n$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h.

$$(d \cdot e) \bmod \phi(n) = 1$$

Der öffentliche Schlüssel ist (n, e) und

der private Schlüssel ist $(p, q, \phi(n), d)$. Wobei $(p, q, \phi(n))$ nicht mehr benötigt und vernichtet werden können.

Verschlüsselung: $f_e(m) = m^e \bmod n = c$

Entschlüsselung: $f_d(c) = c^d \bmod n$

Public-Key Eigenschaft:

Wenn man nur die Zahl n kennt (und nicht etwa die Faktoren p und q oder die Zahl $\phi(x) = (p - 1)(q - 1)$), so kann man aus e nicht d berechnen.



1. Ohne Kenntnis der Primfaktoren p und q kann $\phi(n)$ nicht berechnet werden!
2. Es existiert kein klassischer Algorithmus zur Primfaktorzerlegung

K
A

2048bit ->

~617 Dezimalstellen!!

Signieren von Nachrichten



Bob kann Nachrichten mit seinem privaten Schlüssel signieren.

Es ist egal, ob Bob e oder d als öffentlichen Schlüssel verwendet;
Entschlüsselung/Verschlüsselung funktioniert in beide Richtungen:

Klartext $m=13$

Verschlüsselung: $f_e(m) = m^e \bmod n = 13^7 \bmod 187 = 106$

Entschlüsselung: $f_d(c) = c^d \bmod n = 106^{23} \bmod 187 = 13$

Klartext $m=13$

Verschlüsselung: $f_d(m) = m^d \bmod n = 13^{23} \bmod 187 = 140$

Entschlüsselung: $f_e(c) = c^e \bmod n = 140^7 \bmod 187 = 13$

Signieren von Nachrichten



Bob kann Nachrichten mit seinem privaten Schlüssel signieren.

Es ist egal, ob Bob e oder d als öffentlichen Schlüssel verwendet;
Entschlüsselung/Verschlüsselung funktioniert in beide Richtungen:

Klartext $m=13$

Verschlüsselung: $f_e(m) = m^e \bmod n = 13^7 \bmod 187 = 106$

Entschlüsselung: $f_d(c) = c^d \bmod n = 106^{23} \bmod 187 = 13$

Klartext $m=13$

Verschlüsselung: $f_d(m) = m^d \bmod n = 13^{23} \bmod 187 = 140$

Entschlüsselung: $f_e(c) = c^e \bmod n = 140^7 \bmod 187 = 13$

Idee: Ein mit dem privaten Schlüssel verschlüsselte Text kann mit dem öffentlichen Schlüssel entschlüsselt werden.

Signieren von Nachrichten



Bob kann Nachrichten mit seinem privaten Schlüssel signieren.

Es ist egal, ob Bob e oder d als öffentlichen Schlüssel verwendet;
Entschlüsselung/Verschlüsselung funktioniert in beide Richtungen:

Klartext $m=13$

Verschlüsselung: $f_e(m) = m^e \bmod n = 13^7 \bmod 187 = 106$

Entschlüsselung: $f_d(c) = c^d \bmod n = 106^{23} \bmod 187 = 13$

Klartext $m=13$

Verschlüsselung: $f_d(m) = m^d \bmod n = 13^{23} \bmod 187 = 140$

Entschlüsselung: $f_e(c) = c^e \bmod n = 140^7 \bmod 187 = 13$

Idee: Ein mit dem privaten Schlüssel verschlüsselte Text kann mit dem öffentlichen Schlüssel entschlüsselt werden.

Praktisches Vorgehen: 1) Hash eines Textes berechnen

Signieren von Nachrichten



Bob kann Nachrichten mit seinem privaten Schlüssel signieren.

Es ist egal, ob Bob e oder d als öffentlichen Schlüssel verwendet;
Entschlüsselung/Verschlüsselung funktioniert in beide Richtungen:

Klartext $m=13$

Verschlüsselung: $f_e(m) = m^e \bmod n = 13^7 \bmod 187 = 106$

Entschlüsselung: $f_d(c) = c^d \bmod n = 106^{23} \bmod 187 = 13$

Klartext $m=13$

Verschlüsselung: $f_d(m) = m^d \bmod n = 13^{23} \bmod 187 = 140$

Entschlüsselung: $f_e(c) = c^e \bmod n = 140^7 \bmod 187 = 13$

Idee: Ein mit dem privaten Schlüssel verschlüsselte Text kann mit dem öffentlichen Schlüssel entschlüsselt werden.

Praktisches Vorgehen: 1) Hash eines Textes berechnen

„Eine Hashfunktion ist eine Abbildung, die eine große Eingabemenge, z.B. Dateiinhalte, auf eine kleinere Zielmenge, die Hashwerte, abbildet. [...] Bei Prüfsummen verwendet man Hashwerte, um Übertragungsfehler zu erkennen.“
In der Kryptologie werden spezielle kryptographische Hashfunktionen z.B. zur sicheren Speicherung von Passwörtern verwendet.

Signieren von Nachrichten



Bob kann Nachrichten mit seinem privaten Schlüssel signieren.

Es ist egal, ob Bob e oder d als öffentlichen Schlüssel verwendet;
Entschlüsselung/Verschlüsselung funktioniert in beide Richtungen:

Klartext $m=13$

Verschlüsselung: $f_e(m) = m^e \bmod n = 13^7 \bmod 187 = 106$

Entschlüsselung: $f_d(c) = c^d \bmod n = 106^{23} \bmod 187 = 13$

Klartext $m=13$

Verschlüsselung: $f_d(m) = m^d \bmod n = 13^{23} \bmod 187 = 140$

Entschlüsselung: $f_e(c) = c^e \bmod n = 140^7 \bmod 187 = 13$

Idee: Ein mit dem privaten Schlüssel verschlüsselte Text kann mit dem öffentlichen Schlüssel entschlüsselt werden.

Praktisches Vorgehen: 1) Hash eines Textes berechnen

2) Hash mit dem privaten Schlüssel verschlüsseln

3) verschlüsselter Hash dient als Signatur



Signieren von Nachrichten



Bob kann Nachrichten mit seinem privaten Schlüssel signieren.

Es ist egal, ob Bob e oder d als öffentlichen Schlüssel verwendet;
Entschlüsselung/Verschlüsselung funktioniert in beide Richtungen:

Klartext $m=13$

Verschlüsselung: $f_e(m) = m^e \bmod n = 13^7 \bmod 187 = 106$

Entschlüsselung: $f_d(c) = c^d \bmod n = 106^{23} \bmod 187 = 13$

Klartext $m=13$

Verschlüsselung: $f_d(m) = m^d \bmod n = 13^{23} \bmod 187 = 140$

Entschlüsselung: $f_e(c) = c^e \bmod n = 140^7 \bmod 187 = 13$

Idee: Ein mit dem privaten Schlüssel verschlüsselte Text kann mit dem öffentlichen Schlüssel entschlüsselt werden.

Praktisches Vorgehen: 1) Hash eines Textes berechnen

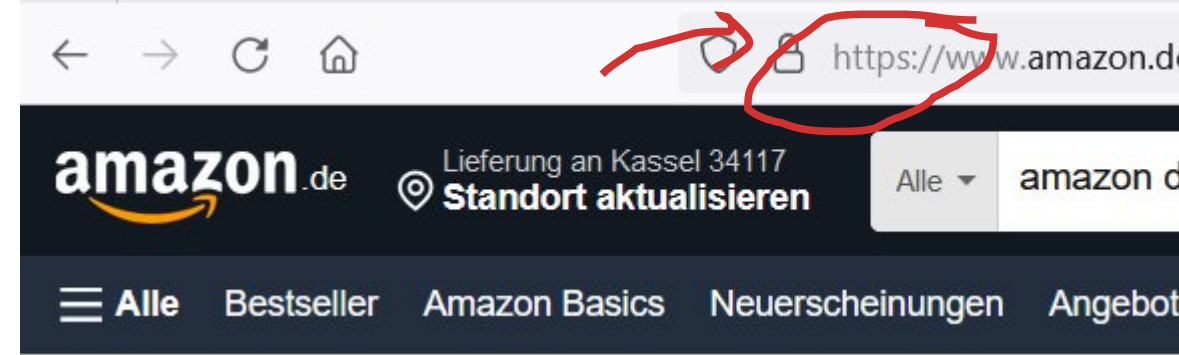
2) Hash mit dem privaten Schlüssel verschlüsseln

3) verschlüsselter Hash dient als Signatur

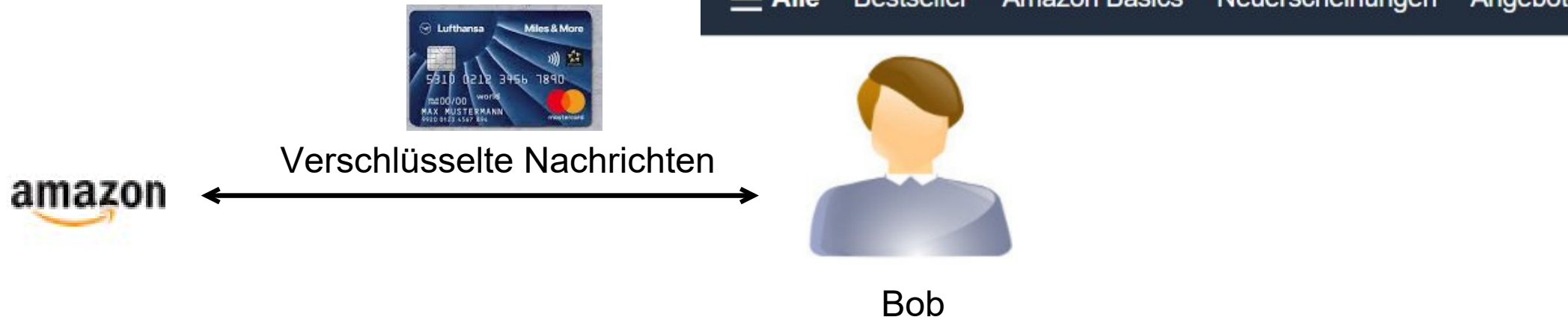
4) Empfänger kann Hash aus dem Text berechnen

und vergleichen mit dem öffentlichen Schlüssel entschlüsselten Hash

HTTPS



HTTPS



„Syntaktisch ist HTTPS identisch mit dem Schema für HTTP, die zusätzliche Verschlüsselung der Daten geschieht mittels SSL/TLS: Unter Verwendung des SSL-Handshake-Protokolls findet zunächst eine geschützte Identifikation und Authentifizierung der Kommunikationspartner statt.

Anschließend wird mit Hilfe **asymmetrischer Verschlüsselung** oder des **Diffie-Hellman-Schlüsselaustauschs** ein gemeinsamer **symmetrischer Sitzungsschlüssel** ausgetauscht. Dieser wird schließlich **zur Verschlüsselung der Nutzdaten verwendet.**“

https://de.wikipedia.org/wiki/Hypertext_Transfer_Protocol_Secure

The basics for a new user

As a new user, you can get started with Bitcoin without understanding the technical details. Once you've installed a Bitcoin wallet on your computer or mobile phone, it will generate your first Bitcoin address and you can create more whenever you need one. You can disclose your addresses to your friends so that they can pay you or vice versa. In fact, this is pretty similar to how email works, except that Bitcoin addresses should be used only once.

The basics for a new user

As a new user, you can get started with Bitcoin without understanding the technical details. Once you've installed a Bitcoin wallet on your computer or mobile phone, it will generate your first Bitcoin address and you can create more whenever you need one. You can disclose your addresses to your friends so that they can pay you or vice versa. In fact, this is pretty similar to how email works, except that Bitcoin addresses should be used only once.

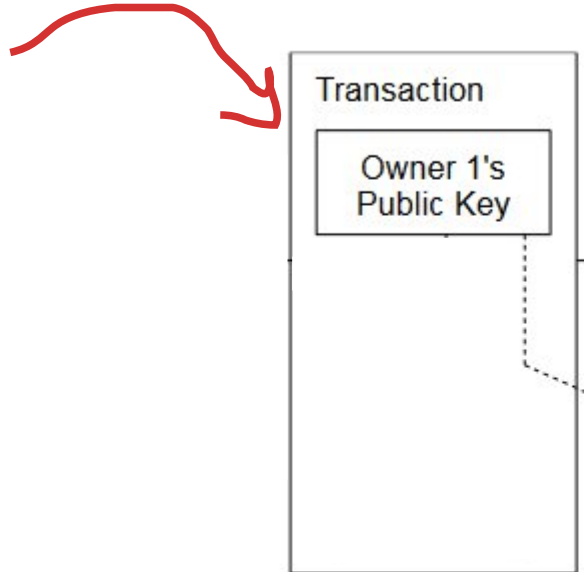


Bitcoins: Transaktionen

Bitcoin: A Peer-to-Peer Electronic Cash System

Satoshi Nakamoto
satoshin@gmx.com
www.bitcoin.org

Information
der letzten
Transaktion:



<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

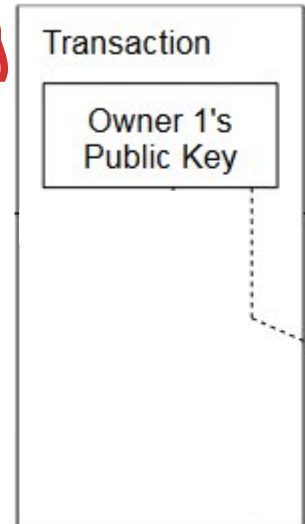
<https://bitcoin.org/en/how-it-works>

Karlsruhe University of Applied Sciences

Bitcoins: Transaktionen



Information
der letzten
Transaktion:



**Owner 1 besitzt
den Bitcoin** (sein
öffentlicher Key ist
enthalten
/gespeichert)

<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

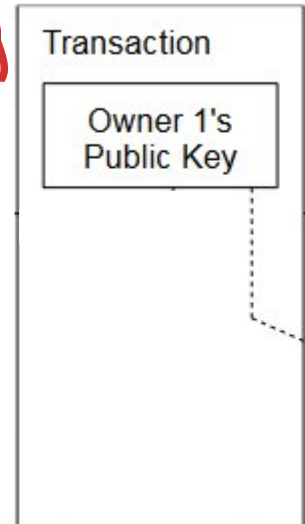
<https://bitcoin.org/en/how-it-works>

Bitcoins: Transaktionen



Information
der letzten
Transaktion:

Owner 1 besitzt
den Bitcoin (sein
öffentlicher Key ist
enthalten
/gespeichert)



Owner 1's
Private Key

Owner 1 möchte seinen Bitcoin verkaufen.
Er benötigt zur authentication seinen privaten Schlüssel!

<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

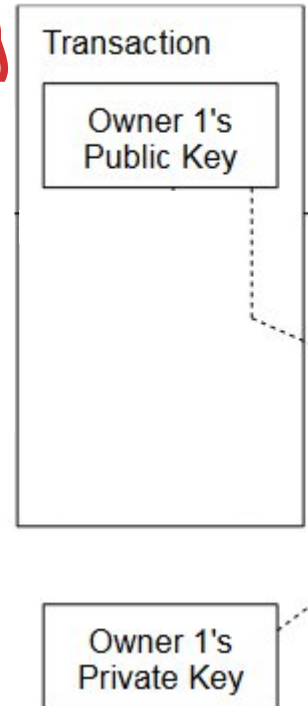
<https://bitcoin.org/en/how-it-works>

Bitcoins: Transaktionen



Information
der letzten
Transaktion:

Owner 1 besitzt
den Bitcoin (sein
öffentlicher Key ist
enthalten
/gespeichert)



<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Karlsruhe University of Applied Sciences

Owner 1 möchte seinen Bitcoin verkaufen.
Er benötigt zur authentication seinen privaten Schlüssel!

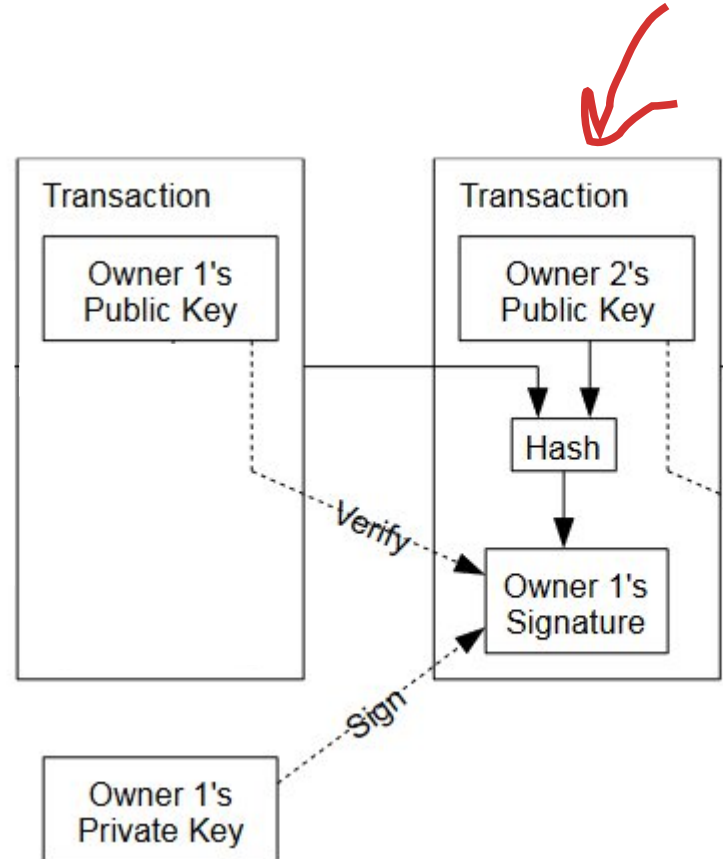
Öffentlicher und privater Schlüssel sind in einer „Wallet“ gespeichert. Für jede Transaktion wird ein neues Schlüsselpaar von der Wallet erzeugt und in dieser gespeichert.



Bitcoins: Transaktionen



Information der letzten Transaktion
Owner 2 besitzt den Bitcoin



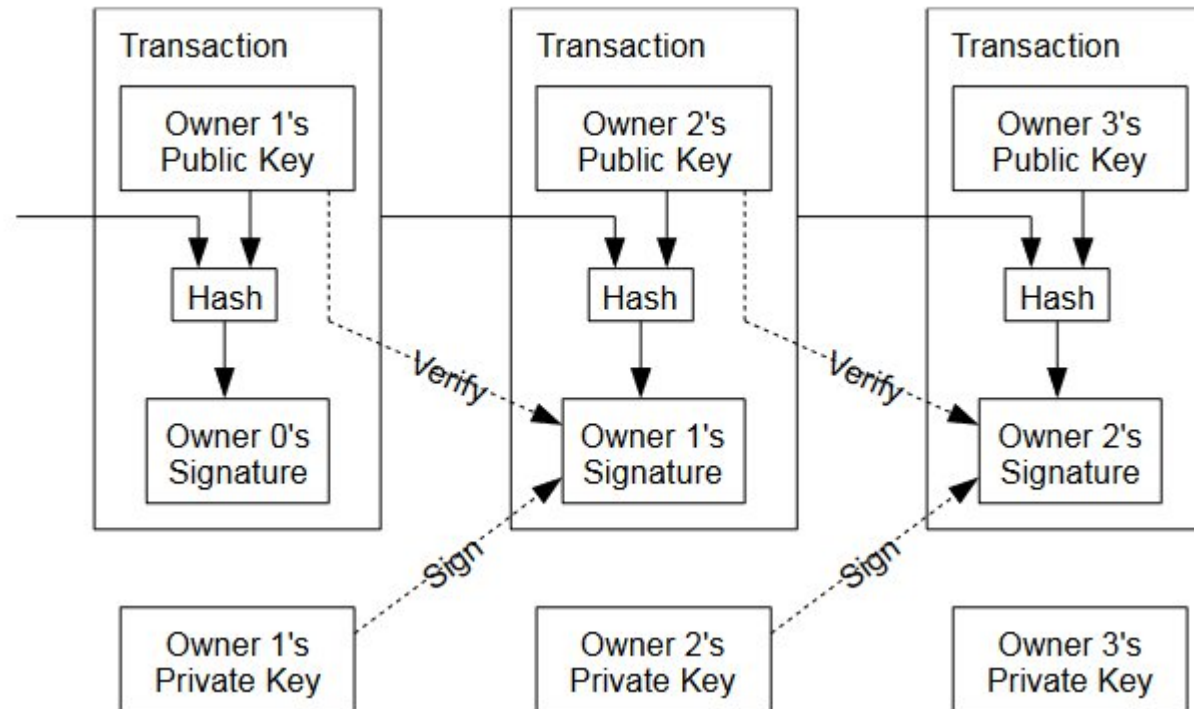
Owner 1 verkauft seinen Bitcoin an Owner 2.

Owner 1 beweist, dass er den Bitcoin besitzt durch den Besitz des privaten Schlüssels!

<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Bitcoins: Transaktionen in einer Kette digitaler Signaturen



Jeder Käufer kann den Besitz durch die Signaturen und die Kette der Signaturen verifizieren.

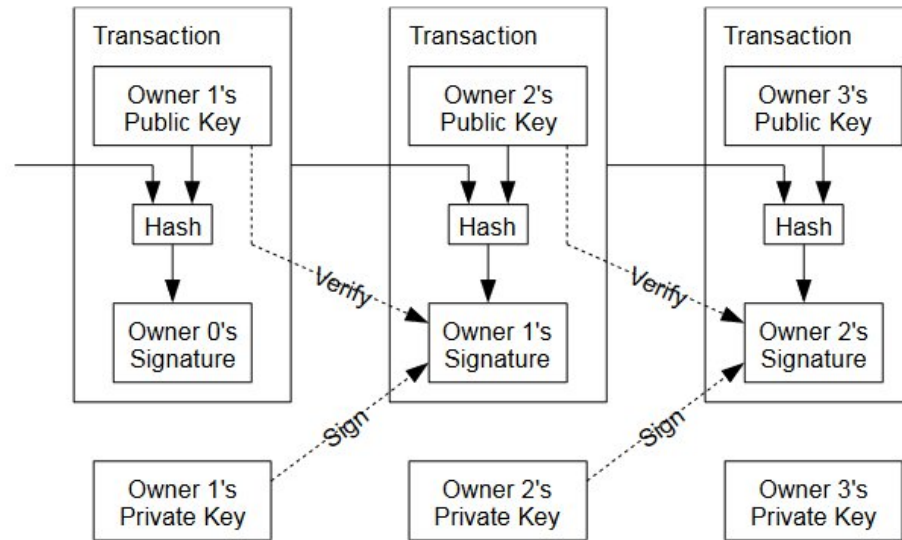
<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Bitcoins: Transaktionen in einer Kette digitaler Signaturen



Problem?



<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Karlsruhe University of Applied Sciences

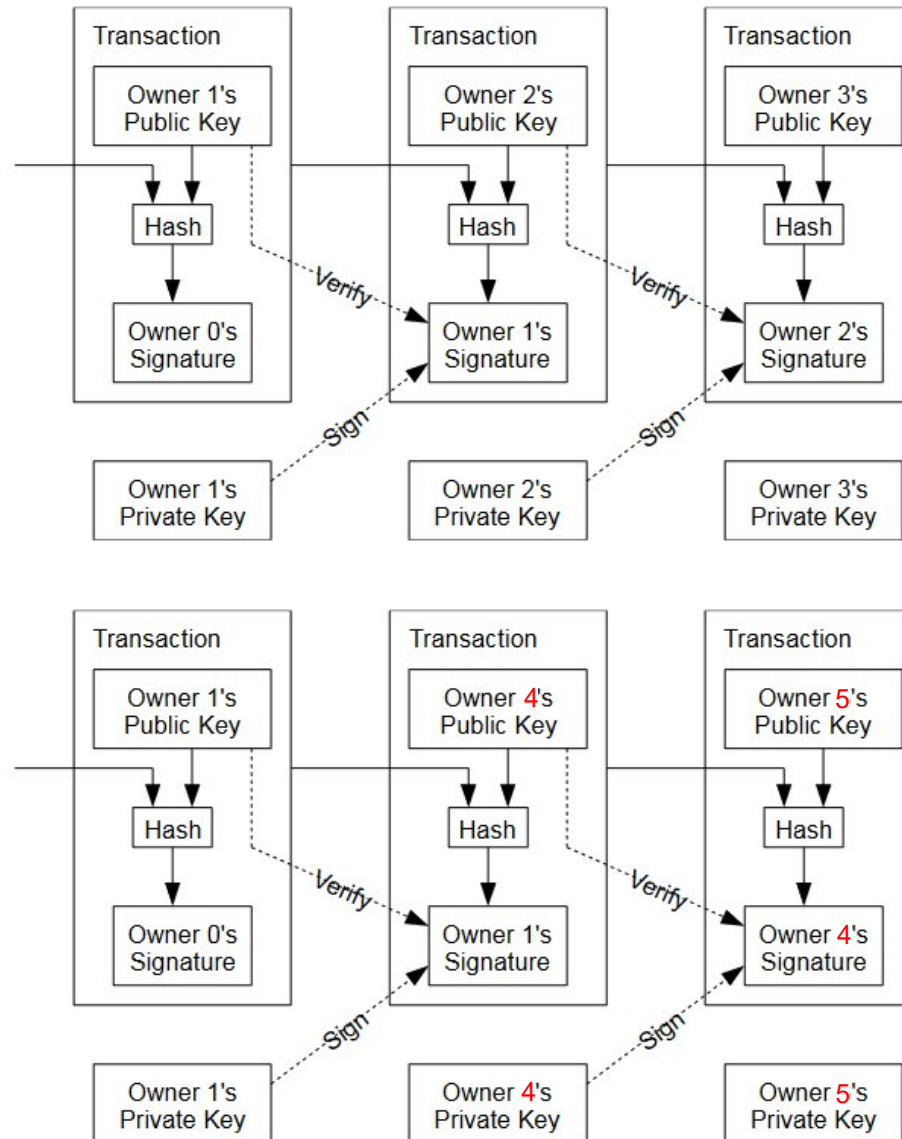


Bitcoins: Transaktionen in einer Kette digitaler Signaturen



Problem:

Wie wird doppeltes Ausgeben verhindert?



<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Bitcoins: Verhindern von Doppeltausgaben



Problem:

Wie wird doppeltes
Ausgeben verhindert?

Traditionell:

- durch ein Institut dem alle Teilnehmer vertrauen (aka Bank)

Bitcoin:

- Jede Transaktion braucht einen Zeitstempel
- Öffentliches Bekanntmachen der Transaktionen
- Die erste Ausgabe des Bitcoins ist die einzig wahre (alle späteren sind Doppeltausgaben!)
- Alle Teilnehmer benutzen ein System, um sich auf eine einzige Transaktionsgeschichte zu einigen

<https://bitcoin.org/bitcoin.pdf>

<https://github.com/bitcoin/bitcoin>

<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Bitcoins: Verhindern von Doppeltausgaben



Problem:

Wie wird doppeltes Ausgeben verhindert?

Einfügen von Zeitinformationen in den Hash

Traditionell:

- durch ein Institut dem alle Teilnehmer vertrauen (aka Bank)

Bitcoin:

- Jede Transaktion braucht einen Zeitstempel
- Öffentliches Bekanntmachen der Transaktionen
- Die erste Ausgabe des Bitcoins ist die einzig wahre (alle späteren sind Doppeltausgaben!)
- Alle Teilnehmer benutzen ein System, um sich auf eine einzige Transaktionsgeschichte zu einigen

<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Bitcoins: Verhindern von Doppeltausgaben



Problem:

Wie wird doppeltes Ausgeben verhindert?

Einfügen von Zeitinformationen in den Hash

Traditionell:

- durch ein Institut dem alle Teilnehmer vertrauen (aka Bank)

Dezentrales Speichern in einem peer-to-peer Netzwerk

Bitcoin:

- Jede Transaktion braucht einen Zeitstempel
- Öffentliches Bekanntmachen der Transaktionen
- Die erste Ausgabe des Bitcoins ist die einzig wahre (alle späteren sind Doppeltausgaben!)
- Alle Teilnehmer benutzen ein System, um sich auf eine einzige Transaktionsgeschichte zu einigen

<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Bitcoins: Verhindern von Doppeltausgaben



Problem:

Wie wird doppeltes Ausgeben verhindert?

Einfügen von Zeitinformationen in den Hash

Traditionell:

- durch ein Institut dem alle Teilnehmer vertrauen (aka Bank)

Dezentrales Speichern in einem peer-to-peer Netzwerk

Bitcoin:

- Jede Transaktion braucht einen Zeitstempel
- Öffentliches Bekanntmachen der Transaktionen
- Die erste Ausgabe des Bitcoins ist die einzig wahre (alle späteren sind Doppeltausgaben!)
- Alle Teilnehmer benutzen ein System, um sich auf eine einzige Transaktionsgeschichte zu einigen

<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Karlsruhe University of Applied Sciences

Wieso ist das nötig?



Bitcoins: Verhindern von Doppeltausgaben



Problem:

Wie wird doppeltes Ausgeben verhindert?

Einfügen von Zeitinformationen in den Hash

Traditionell:

- durch ein Institut dem alle Teilnehmer vertrauen (aka Bank)

Dezentrales Speichern in einem peer-to-peer Netzwerk

Bitcoin:

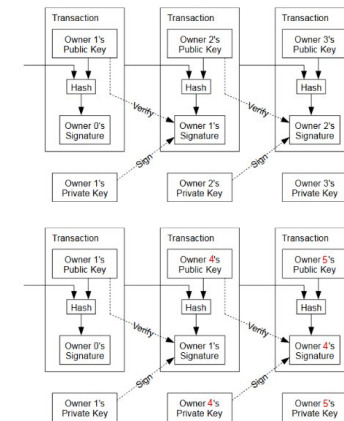
- Jede Transaktion braucht einen Zeitstempel
- Öffentliches Bekanntmachen der Transaktionen
- Die erste Ausgabe des Bitcoins ist die einzig wahre (alle späteren sind Doppeltausgaben!)
- Alle Teilnehmer benutzen ein System, um sich auf eine einzige Transaktionsgeschichte zu einigen

<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Karlsruhe University of Applied Sciences

Ansonsten könnten viele Ketten nebeneinander existieren...



Bitcoins: Verhindern von Doppeltausgaben



Problem:

Wie wird doppeltes Ausgeben verhindert?

Einfügen von Zeitinformationen in den Hash

Traditionell:

- durch ein Institut dem alle Teilnehmer vertrauen (aka Bank)

Dezentrales Speichern in einem peer-to-peer Netzwerk

Bitcoin:

- Jede Transaktion braucht einen Zeitstempel
- Öffentliches Bekanntmachen der Transaktionen
- Die erste Ausgabe des Bitcoins ist die einzig wahre (alle späteren sind Doppeltausgaben!)
- Alle Teilnehmer benutzen ein System, um sich auf eine einzige Transaktionsgeschichte zu einigen

<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Karlsruhe University of Applied Sciences

Blockchain



Bitcoins: Blockchain



- Block auf Informationen einer oder mehrerer Transaktionen
- Jeder Block braucht einen Zeitstempel (time server)
- Anhängen eines neuen Blocks an eine Kette erfordert Arbeit (proof-of-work)

<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>



Bitcoins: Blockchain



- Block auf Informationen einer oder mehrerer Transaktionen
- Jeder Block braucht einen Zeitstempel (time server)
- Anhängen eines neuen Blocks an eine Kette erfordert Arbeit (proof-of-work)



- Erschwert das Erstellen einer alternativen Kette
- Erschwert das Abändern (Fälschen) der letzten Blöcke einer Kette (hierzu müsste für jedes Blockelement die Arbeit erneut aufgebracht werden)

<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Bitcoins: Blockchain



- Block auf Informationen einer oder mehrerer Transaktionen
- Jeder Block braucht einen Zeitstempel (time server)
- Anhängen eines neuen Blocks an eine Kette erfordert Arbeit (proof-of-work)
- Die längste Kette (die mit der meisten Arbeit) ist diejenige auf die sich alle einigen

<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

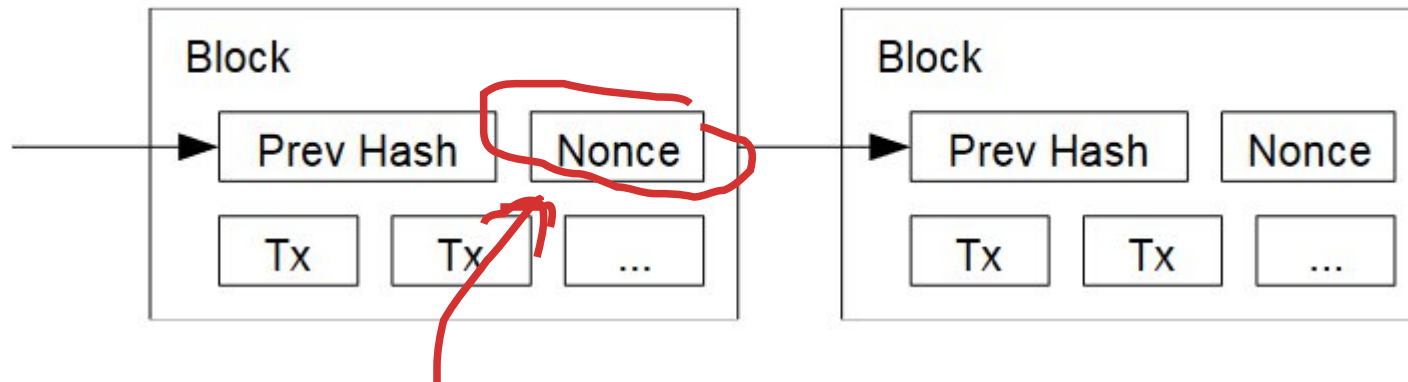
<https://bitcoin.org/en/how-it-works>



Bitcoins: Blockchain (proof-of-work)



- Block auf Informationen einer oder mehrerer Transaktionen
- Jeder Block braucht einen Zeitstempel (time server)
- Anhängen eines neuen Blocks an eine Kette erfordert Arbeit (proof-of-work)
- Die längste Kette (die mit der meisten Arbeit) ist diejenige auf die sich alle einigen



proof of work besteht darin einen Wert (Nonce) zu finden, für den der Hash des Blockes eine bestimmte Eigenschaft hat. Bei Bitcoins eine vorgegebene Anzahl von führenden Nullen.

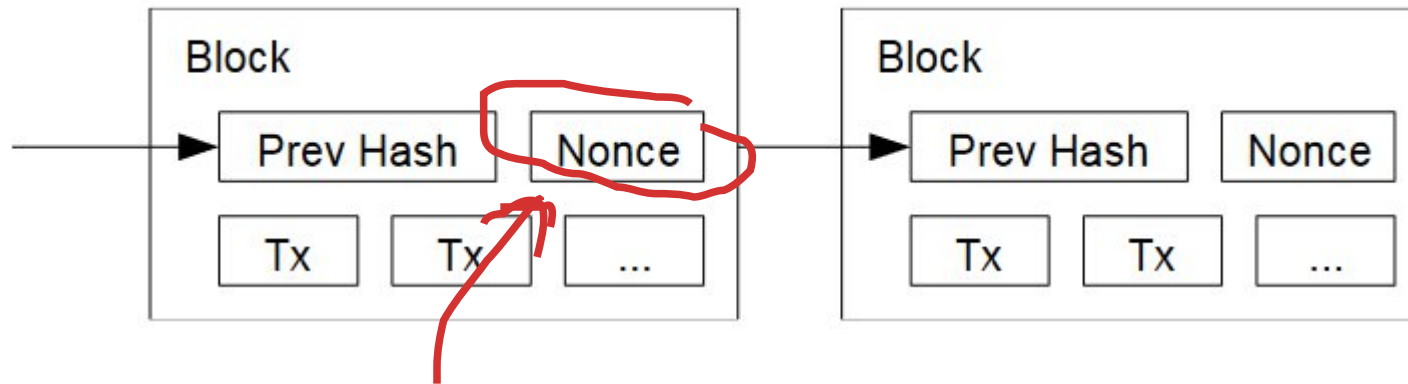
<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Bitcoins: Blockchain (proof-of-work)



- Block auf Informationen einer oder mehrerer Transaktionen
- Jeder Block braucht einen Zeitstempel (time server)
- Anhängen eines neuen Blocks an eine Kette erfordert Arbeit (proof-of-work)
- Die längste Kette (die mit der meisten Arbeit) ist diejenige auf die sich alle einigen



proof of work besteht darin einen Wert (Nonce) zu finden, für den der Hash des Blockes eine bestimmte Eigenschaft hat. Bei Bitcoins eine vorgegebene Anzahl von führenden Nullen.

<https://bitcoin.org/bitcoin.pdf>
<https://github.com/bitcoin/bitcoin>
<https://developer.bitcoin.org/devguide/index.html>

<https://bitcoin.org/en/how-it-works>

Karlsruhe University of Applied Sciences

„To compensate for increasing hardware speed and varying interest in running nodes over time, the proof-of-work difficulty is determined by a moving average targeting an average number of blocks per hour. If they're generated too fast, the difficulty increases“

6. Incentive

By convention, the first transaction in a block is a special transaction that starts a new coin owned by the creator of the block. This adds an incentive for nodes to support the network, and provides a way to initially distribute coins into circulation, since there is no central authority to issue them. The steady addition of a constant amount of new coins is analogous to gold miners expending resources to add gold to circulation. In our case, it is CPU time and electricity that is expended.

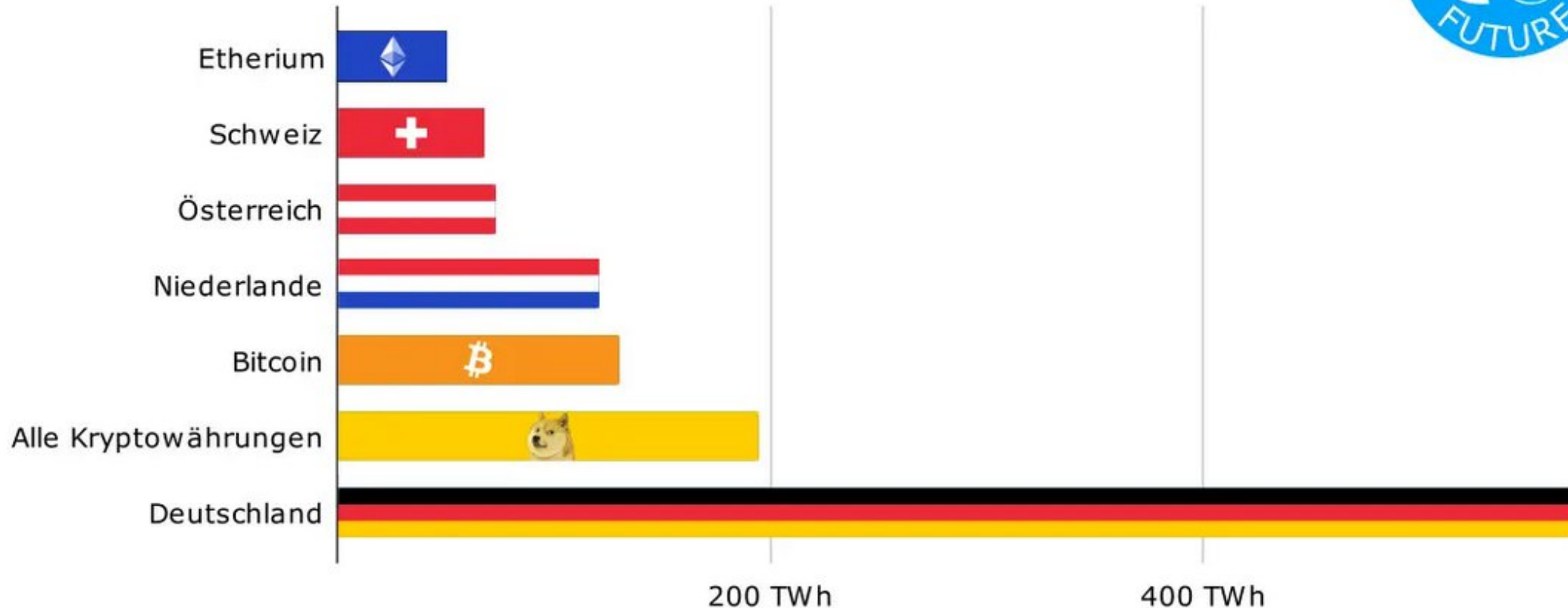
The incentive can also be funded with transaction fees. If the output value of a transaction is less than its input value, the difference is a transaction fee that is added to the incentive value of the block containing the transaction. Once a predetermined number of coins have entered circulation, the incentive can transition entirely to transaction fees and be completely inflation free.

The incentive may help encourage nodes to stay honest. If a greedy attacker is able to assemble more CPU power than all the honest nodes, he would have to choose between using it to defraud people by stealing back his payments, or using it to generate new coins. He ought to find it more profitable to play by the rules, such rules that favour him with more new coins than everyone else combined, than to undermine the system and the validity of his own wealth.

Bitcoins: Proof-of-work energy consumption...



Stromverbrauch von Kryptowährungen
in TWh im Jahr 2020 verglichen mit ausgewählten Ländern



Bitcoins: Proof-of-work energy consumption...



HEIZEN MIT STROM

+ Heizlüfter mit Bitcoin-Miner: Ofen Pro von 21energy im Test

Die Wohnung gemütlich heizen und damit Geld verdienen? Der Ofen Pro von 21energy schürft Bitcoins und gibt dabei Wärme ab. So sollen die Heizkosten sinken.

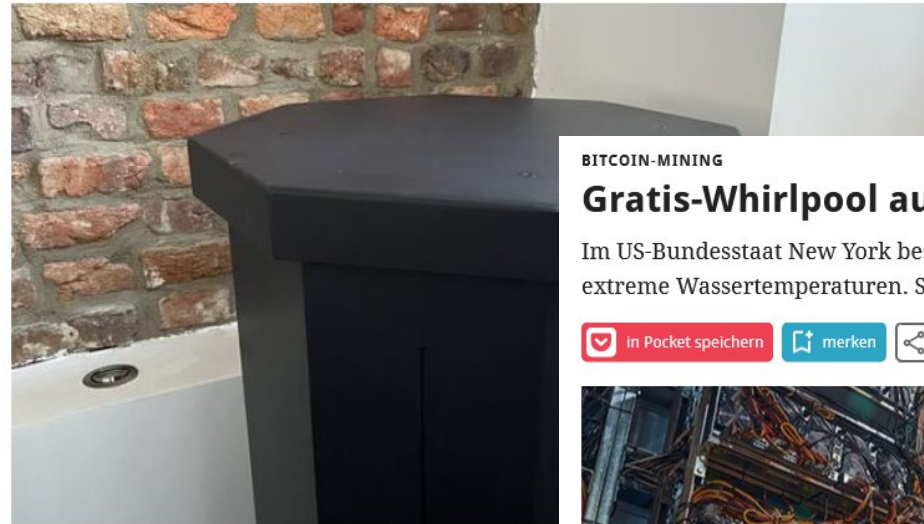
Artikel verschenken **NEU**

👍 🔊 🖨️ 💬 163



Elektroheizungen und Bitcoin Mining als sinnvolle Alternativen?

von Sören Imöhl
24. Januar 2024



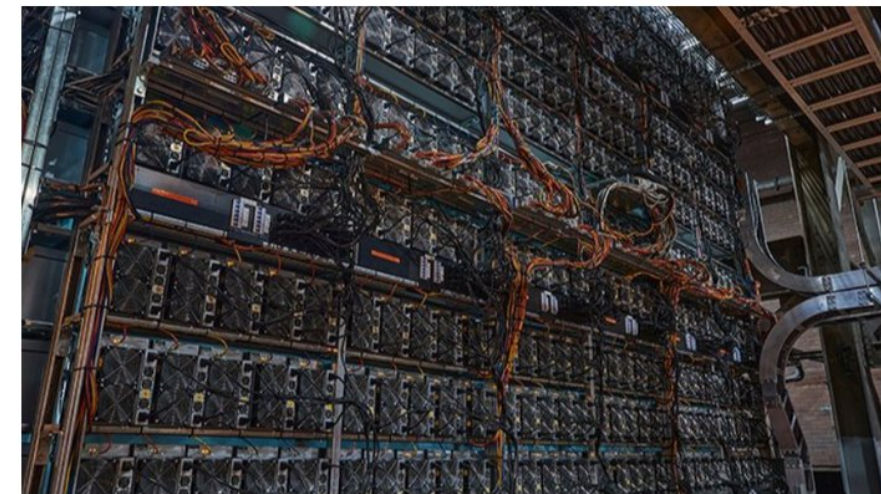
BITCOIN-MINING

Gratis-Whirlpool auf Kosten der Umwelt

Im US-Bundesstaat New York beschweren sich Anwohner des Seneca Lake über extreme Wassertemperaturen. Schuld ist eine **Bitcoin-Farm** am Ufer.

in Pocket speichern merken

6. Juli 2021, 12:48 Uhr, Christian Hensen



Unheimliche Temperaturen für ein Ventilator. Es sieht so bei Bitcoin aus

<https://www.heise.de/tests/Heizluefter-mit-Bitcoin-Miner-Ofen-Pro-von-21energy-im-Test-9968169.html>

<https://www.golem.de/news/bitcoin-mining-gratis-whirlpool-auf-kosten-der-umwelt-2107-157907.html>

<https://www.wiwo.de/finanzen/immobilien/heizen-mit-strom-elektroheizungen-und-bitcoin-mining-als-sinnvolle-alternativen/29417534.html>

Bitcoins: Privacy



The traditional banking model achieves a level of privacy by limiting access to information to the parties involved and the trusted third party. The necessity to announce all transactions publicly precludes this method, but privacy can still be maintained by breaking the flow of information in another place: by keeping public keys anonymous. The public can see that someone is sending an amount to someone else, but without information linking the transaction to anyone. This is similar to the level of information released by stock exchanges, where the time and size of individual trades, the "tape", is made public, but without telling who the parties were.

Traditional Privacy Model



New Privacy Model





Potenz-Modulo Rechnen - Beispielrechnungen



Potenz-Modulo Rechnung

$$8 \bmod 3 = 2 \quad 2^3 \bmod 3 = 2$$

$$\begin{aligned} 8:3 &= 2 \quad 200^{300} \bmod 3 \\ \frac{6}{2} & \quad a^n \bmod k \quad (200)^{2 \cdot 150} \\ &= (a \bmod k)^n \bmod k = (200^2)^{150} \end{aligned}$$

$$\begin{aligned} 200^{300} \bmod 3 &= (200 \bmod 3)^{300} \bmod 3 \\ &= 2^{300} \bmod 3 = (2^2)^{150} \bmod 3 \\ &= 2^{2 \cdot 150} \bmod 3 \end{aligned}$$

$200:3=66$
 $\frac{198}{2}$

<https://youtu.be/ribhv53H288>
<https://youtu.be/tLL5zkURcBs>
<https://youtu.be/k6d29PTHtml>

Den Rest großer Potenzen mit „Modularem Potenzieren“ berechnen

Beim Potenzieren von zwei Zahlen entstehen schnell große Zahlen, mit denen das Rechnen mühsam oder, wenn die Anzeige des Taschenrechners sie nicht mehr vollständig anzeigen lässt, unmöglich wird. Soll jedoch der **Rest** einer solchen Potenz berechnet werden kann man die Rechnung mit Hilfe von zwei Tricks vereinfachen:

(Regel I) Lässt sich der Exponent als **Summe** zweier kleinerer Zahlen darstellen, so gilt:

$$x^{a+b} = x^a \cdot x^b$$

Soll der Rest der Potenz gebildet werden, so kann die Restfunktion bereits auf Zwischenergebnisse angewendet werden, denn bezüglich der Restbildung zur Division durch d gilt:

$$x^{a+b} \bmod d = ((x^a \bmod d) \cdot (x^b \bmod d)) \bmod d$$

(Regel II) Lässt sich der Exponent als **Produkt** zweier kleinerer Zahlen darstellen, so gilt:

$$x^{a \cdot b} = (x^a)^b$$

Soll der Rest der Potenz gebildet werden, so kann die Restfunktion bereits auf Zwischenergebnisse angewendet werden, denn bezüglich der Restbildung zur Division durch d gilt:

$$x^{a \cdot b} \bmod d = (x^a \bmod d)^b \bmod d$$

Wie führt ein Computer modulares Potenzieren durch?

Wir haben zur Vereinfachung der Berechnung einige willkürliche Entscheidungen getroffen. Will man das Verfahren für einen Computer programmieren, so braucht man ein Verfahren, das immer zu einem Ergebnis kommt. Ein Ansatz wäre, zunächst zu prüfen, ob der Exponent gerade ist oder ungerade. Ist er ungerade, so kann Regel I angewendet werden, um x^p in $x^1 \cdot x^{p-1}$ zu wandeln. Anschließend kann so lange der Exponent nach Regel II durch 2 geteilt werden, bis er den Wert 1 annimmt:

$$x^5 = x^1 \cdot (x^1)^2$$

https://medienwissenschaft.uni-bayreuth.de/inik/material/email_nur_fuer_dich/3_verschluesseln/3.3_asymmetrisch_verschluesseln/Modulares%20Potenzieren%20-%20AB.pdf
https://medienwissenschaft.uni-bayreuth.de/inik/material/email_nur_fuer_dich/3_verschluesseln/3.3_asymmetrisch_verschluesseln/

Euklidische Algorithmus



Dazu brauchen wir den **Euklidischen Algorithmus** aus Lektion 4. Mit diesem Algorithmus haben wir den größten gemeinsamen Teiler zweier Zahlen a und b mit $a > b$, also $\text{ggT}(a, b)$, berechnet:

Gegeben sind zwei positive ganze Zahlen a und b mit $a > b$.

- 1. Solange $a \neq 0$ ist, wiederhole die Operationen 2.–3.*
- 2. Ersetze a durch $a \bmod b$.*
- 3. Vertausche die beiden Zahlen a und b .*
- 4. Der größte gemeinsame Teiler ist die Zahl b .*

$$\begin{aligned}\text{ggT}(7, 5) &\stackrel{(T8)}{=} \text{ggT}(2, 5) \\ &\stackrel{(T1)}{=} \text{ggT}(5, 2) \\ &\stackrel{(T8)}{=} \text{ggT}(1, 2) \\ &\stackrel{(T1)}{=} \text{ggT}(2, 1) \\ &\stackrel{(T8)}{=} \text{ggT}(0, 1) \\ &\stackrel{(T1)}{=} \text{ggT}(1, 0) = 1\end{aligned}$$

Erweiterter Euklidischer Algorithmus



erweiterten Euklidischen Algorithmus. Der Algorithmus funktioniert genau gleich wie der Euklidische Algorithmus, nur dass man die Ausgangszahlen a und b durch die entsprechende ganzzahlige Linearkombination ersetzt. Wir ersetzen also am Anfang a durch

$$1 \cdot a + 0 \cdot b$$

und b durch

$$0 \cdot a + 1 \cdot b.$$

Dann rechnen wir gleich weiter wie beim Euklidischen Algorithmus, nur dass wir in jedem Schritt die Zahlen als ganzzahlige Linearkombination von a und b stehen lassen. Dann bekommen wir am Schluss den größten gemeinsamen Teiler von a und b , ausgedrückt durch eine ganzzahlige Linearkombination von a und b .

Da gilt:

Es sei t der größte gemeinsame Teiler zweier Zahlen $a, b \in \mathbb{N}$. Dann gibt es zwei Zahlen $\alpha, \beta \in \mathbb{Z}$, so dass

$$t = \alpha \cdot a + \beta \cdot b$$

*gilt. Man nennt $\alpha \cdot a + \beta \cdot b$ eine **ganzzahlige Linearkombination** von a und b .*

Erweiterter Euklidischer Algorithmus



Wir wissen nun, dass wir mit dem erweiterten Euklidischen Algorithmus nicht nur den größten gemeinsamen Teiler t zweier Zahlen a und b berechnen können. Wir können t sogar als Linearkombination von a und b darstellen:

$$t = \alpha \cdot a + \beta \cdot b.$$

Wie können wir nun aber diesen erweiterten Euklidischen Algorithmus zum Berechnen von Inversen verwenden? Wir möchten die multiplikative Inverse von w modulo m berechnen. Wir haben vorausgesetzt, dass w und m teilerfremd sind. Also würde der Euklidische Algorithmus den größten gemeinsamen Teiler $t = 1$ finden. Mit dem erweiterten Euklidischen Algorithmus können wir außerdem 1 als eine Linearkombination von w und m darstellen:

$$1 = \alpha \cdot w + \beta \cdot m.$$

Wenn wir diese Gleichung modulo m rechnen, fällt der zweite Summand auf der rechten Seite weg:

$$1 = \alpha \cdot w \bmod m.$$

Das α ist nun die multiplikative Inverse von w modulo m . Wir haben also Folgendes gezeigt:

Die Zahl w besitzt eine multiplikative Inverse modulo m , wenn w und m teilerfremd sind. Außerdem liefert uns der erweiterte Euklidische Algorithmus ein Verfahren, mit welchem wir diese multiplikative Inverse effizient berechnen können.